

MANager for SoC Test

SIT: Simplified ICL Tree



Revision History

16/05/2023 : Updated for Open Source release of MAST

26/05/2020: Updated to P1687.1

5/10/2017: Created



Index

1	Introduction	1
2	SIT Building blocks	1
2.1	Hierarchy	1
2.2	Implicit connections	2
2.2.1	Scan, Data elements and ScanInterfaces	3
2.3	Registers and Instruments	4
2.4	Reconfigurable Scan Path elements.....	4
2.5	Black Boxes	6
2.6	Instances and file organization	6
2.7	Aliasing	6
2.8	Parameters	6
2.9	Gating on control signals and clock routing.....	6
2.10	PDL integration	7
3	Path Selectors	8
4	AccessInterfaces	10
5	Translators.....	12
6	SIT support for custom modules	14
7	Complete SIT Syntax (from Doxygen) OUTDATED, REFER TO REPOSITORY	15
7.1	LEAF NODES	15
7.2	Parent NODES.....	16
7.2.1	CHAINED hierarchy	17
7.2.2	LINKED hierarchy.....	17



7.3	USAGE.....	22
8	Examples from the IEEE 1687-2014 Standard Document	24
8.1	Annex E, Example 1	24
8.2	Annex E, Example 2	26
8.3	Annex E, Example 3	26
8.4	Annex E, Example 4	28
8.5	Annex E, Example 5	29
8.6	Annex E, Example 6	30
8.7	Annex E, Example 7	31
8.8	Annex E, Example 8	33
8.9	Annex E, Example 9	34
8.10	Annex E, Example 10	36
8.11	Annex E, Example 11	38
8.12	Annex E, Example 12	40
8.13	Annex E, Example 13	41
8.14	Annex E, Example 14	43
8.15	Annex E, Example 15	44
8.16	Annex E, Example 16	44
8.17	Annex E, Example 17	44
8.18	Annex E, Example 18	44
8.19	Annex E, Example 19	45
8.20	Annex E, Example 20	47
8.21	Annex E, Example 21	50



8.22	Annex E, Example 22	52
8.23	Annex E, Example 23	54
8.24	Annex E, Example 24	56
8.25	Annex E, Example 25	58
8.26	Annex E, Example 26	59
8.27	Annex E, Example 27	62
8.28	Annex E, Example 28	62
8.29	Annex E, Example 29	62
8.30	Annex E, Example 30	64
8.31	Annex E, Example 31	67
9	References	69



1 Introduction

The official language of the IEEE 1687-2014 is the Instrument Connectivity Language, or ICL. It has been developed for several reasons, the main being:

- Express hierarchical designs;
- Describe the details of the connectivity

To achieve these goals, ICL is basically a structural language: it is possible to instantiate elements hierarchically, define communication ports on them and express their connection. The resulting description is extremely detailed and really close to a structural RTL description. This means that writing an ICL description is a difficult exercise, usually done after a preliminary RTL description has been established to assure coherence between the two. Moreover, the structural nature of the language makes it difficult to express some constructs. Last but not least, an important part of the information conveyed by ICL, and which makes its complexity, is rather connected to ATPG than to retargeting, and it is a replica of information already available in the RTL or implied by the underlying IEEE 1149.1 standard.

For all these reasons, MAST allows systems to be described in a format, the Simplified ICL Tree (SIT), which conveys only the minimal information set required for retargeting. A SIT file is a direct transposition of the hierarchical structure of the system, while the connectivity is described in a functional way. This allows for an extremely concise and lightweight description that can be established from the high-level specification of the system, with little or no information replication with RTL. This also means that once established at specification time, a SIT description is completely portable during the development cycle of the system, all the way, if needed, to vector generation or ATE execution.

In this document, we will first briefly present the main building blocks and syntax elements of a SIT file, pointing out their ICL equivalents, and then propose the SIT description of examples taken from the Annex E of the IEEE 1687-2014 standard document. The precise rules of the language are then provided as an Annex.

2 SIT Building blocks

2.1 Hierarchy

As its name suggests, a SIT file depicts a system hierarchy in terms of a tree. So, if module B is instantiated inside module A, a SIT will depict B as a “child” of the “root node” A, as depicted in



Figure 1. This a straightforward yet efficient representation, as it expressed the full hierarchical relationship without any assumption on how this is actually achieved.

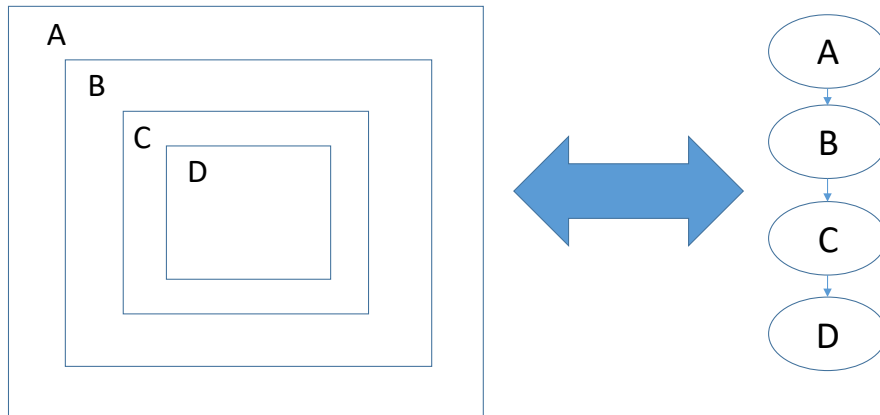


Figure 1 Tree representation of a hierarchical system

In a SIT file, this will be represented as follows:

- (1) CHAIN A
- (2) {
- (3) CHAIN B
- (4) { CHAIN C
- (5) { CHAIN D}
- (6) }}

The keyword “CHAIN”, followed by a string identifies a hierarchical component, while the accolades ‘{’ and ‘}’ contain the lower-level elements. Unnamed hierarchical levels can be defined by using only the accolades

Please note that space, tabulations and newlines can be used freely to provide the desired indentation.

2.2 Implicit connections

The role of a retargeter is to compose vector segments of a given hierarchical level to obtain the higher-level vector. To achieve this, it does not really need full connectivity information, as

provided by ICL, but just the order in which the composition must be done, as depicted in Figure 2

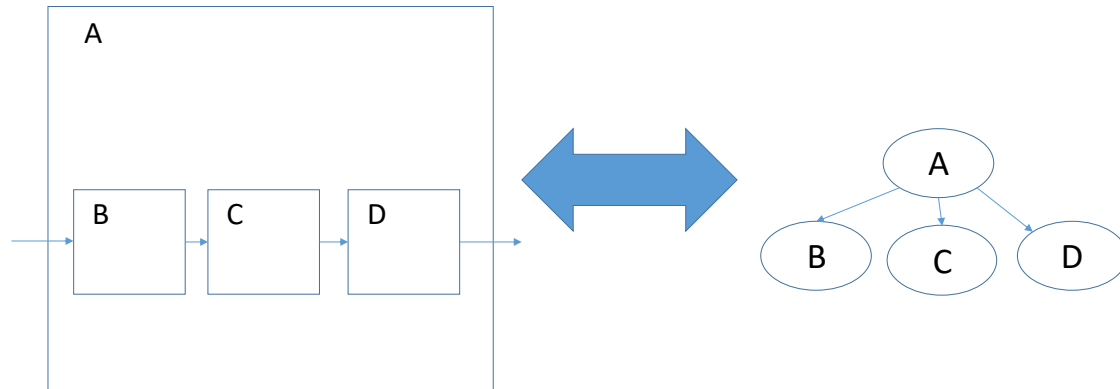


Figure 2 Implicit connections

In a SIT file, this will be represented as follows:

- (1) CHAIN A
- (2) {
- (3) INSTANCE B OF my_element
- (4) CHAIN C
- (5) { ... }
- (6) INSTANCE D OF my_element
- (7) }

All nodes between accolades are considered are composing an ordered chain. In SIT, this is called a “Chained hierarchy”. In lines 3 and 6 we used an instance rule (please see Section 0 for details).

2.2.1 Scan, Data elements and ScanInterfaces

In ICL there is a clear distinction between “Scan” and “Data” paths, as only the former can be connected to ScanInterface and eventually to an 1149.1 AccesLink. This distinction is not needed in SIT for two reasons:

- SIT implements the AccessInterface abstraction [2], so the connection type is defined by its top-level AccessInterface (see Section 4);



- Connections are implicit, all elements on the same chain share the connection defined by the overlying AccessInterface.

2.3 Registers and Instruments

In ICL there is the concept of “Raw instrument”, on whose boundaries can be defined PDL operations, and it is possible to define connections to one or more “ScanRegister” to be accessed from an AccessLink, and which are the final targets of any retargeting information. For this reason, SIT simply defines some REGISTERS which are the final targets of PDL retargeting, as depicted in Figure 3. Please note that SIT has provisions for information such as the default load and reset values (ref. Section 0).

Aliasing can be used to boost PDL portability between ICL and SIT (ref. Section 2.7)

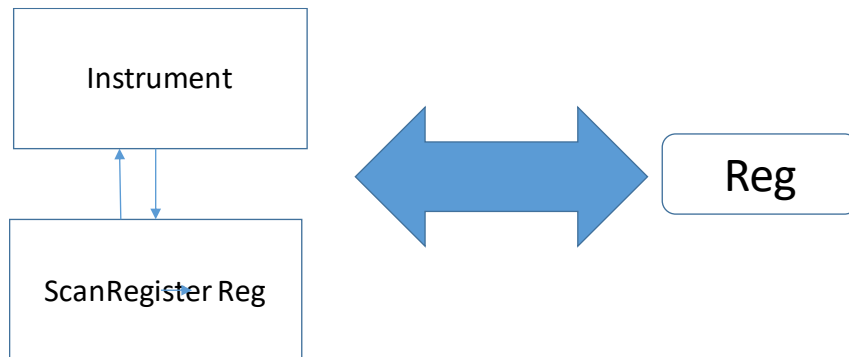


Figure 3 Tree representation of a ScanRegister+Instrument

In a SIT file, this will be simply represented as follows, where 12 is the size and “Bypass” the “default_LoadValue”:

(1) REGISTER reg 12 Bypass: "0xABC"

2.4 Reconfigurable Scan Path elements

In ICL, the only element able to dynamically modify a scan path is the ScanMux. Faithful to the overall structural approach, it is exactly what its name implies: a simple multiplexer, with inputs, control signals and an explicit truth table. SIT takes a completely different approach and describes its functional behavior in Object-Oriented terms: a ScanMux is a “LINKER”, i.e. a node whose hierarchical behavior is defined by a “PathSelector”, i.e. a standardized collection of methods describing its behavior. Its complete behavior is described in [1], but the principle is simple: if the ScanMux is closed (i.e. “deselected”) it is considered as a leaf node. If on the other hand the ScanMux is open (i.e. “selected”), then it is considered as a “CHAIN” node with one or more sub-modules. Figure 4 shows how a typical SIB can be seen in these terms.

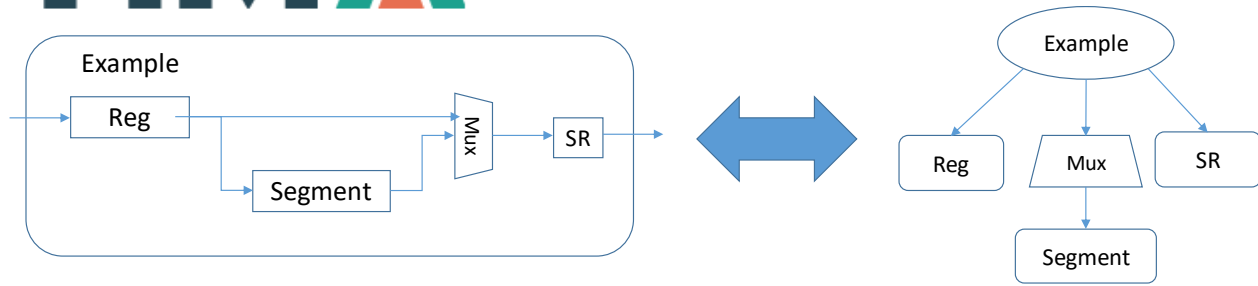


Figure 4 Tree Representation of a SIB

In SIT terms, “Segment” belongs to a lower hierarchical level, whose access is controlled by the status of the selection element, “Mux”. Therefore the syntax is similar as per chained hierarchies (section 2.1), but parenthesis ‘(’ and ‘)’ are used to encapsulate the lower level accessed through the LINKER: it is a “Linked hierarchy”

A Possible SIT description is:

- (1) CHAIN Example
- (2) {
- (3) REGISTER Reg 8 Bypass: "0x00"
- (4) LINKER Mux Binary SR 1
- (5) (REGISTER Segment 12 Bypass: "0xABC")
- (6) REGISTER SR 1 Bypass: "0x0"
- (7) }

In line (4), a LINKER is declared having 1 possible hierarchical derivation, and being selected by the node SR. The main difference from ICL is that instead of having a compulsory explicit truth table, the select behavior can also be described using common coding schemes. More details on Path Selectors are given in Section

Please note that SIT provides macros for usual constructs like SIBs and MIBs (cfr Section 0), allowing for a more compact syntax:

- (8) CHAIN Example
- (9) {
- (10) REGISTER 8 Bypass: "0x00"



(11) SIB sib POST HIGH

(12) (REGISTER Segment 12 Bypass: "0xABC")

(13) }

LINKER does not only provides for a much more powerful and flexible description than ScanRegister (as detailed in Section 0 and [1]), but also for a extremely compact and easily readable description, making creation, validation and maintenance of SIT models extremely simple.

The fact of using different separators for chained and linked hierarchies improves significantly the readability and maintainability of SIT files.

2.5 Black Boxes

MAST supports the definition of a Black Box element which can be the target of iScan operations. The length of its internal Scan Chain is deduced from the last vector it has been the target of.

<NOT implemented yet in SIT Parser, but supported by MAST Kernel>

2.6 Instances and file organization

Each SIT file describes only one module, but it possible to instantiate modules defined in another SIT file to boost code reutilization and readability. Details can be found in Section 7.

2.7 Aliasing

<NOT implemented yet, will follow the same rules as ICL>

2.8 Parameters

<NOT implemented yet, will follow the same rules as ICL>

2.9 Gating on control signals and clock routing

ICL allows for gating control signals (UpdateEnable, CaptureEnable, etc..), as well as an explicit routing of the clock signals. These are low-level features that strongly depends on the RTL implementation, so they are not supported by SIT which is focused on high-level modelling

In the IEEE 1687-2014 standard document the association between ICL and PDL is done through the “iProcForModule” PDL command which, as its name states, makes an explicit reference to the ICL module they can be applied to. This is because the principle is that a module is delivered with a set of functions, and the user will compose a top-level routine by explicitly calling the iProcs.

While still supporting this usage, MAST proposes a different approach: a module can be delivered with one or more PDL functions associated with it. When launched, MAST will start and schedule them concurrently: execute will finish when all sub-functions have exited. In this context, the “iProcForModule” command not needed as the execution context is implicitly known.

Please refer to Section 7 for the corresponding SIT rules, and to the MAST User Manual for details on MAST Execution model.

3 Path Selectors

Because of its structural nature, ICL provides only two hierarchy-enabling components: the Mux and the OneHotGroup, both declined in terms of Scan or Data registers. The truth table of the mux selection must be explicitly written, optionally using LogicSignal to express more complex Boolean rules. This can rapidly become cumbersome and difficult to validate, and the LogicSignal syntax will not necessarily mirror the underlying RTL.

To simplify the issue, MAST uses a functional approach: a LINKER is an extension of a Mux, i.e. it is node that enables to select one or more lower hierarchical levels. Instead of explicitly define the truth table, a Linker uses a standardized API called PathSelector that expresses the conditions for this selection. SIT provides a set of defaults PathSelectors that can be directly used without having to know the implementation details, which are available in [1].

Figure 5 depicts the example we will use in this section: a Mux controlling the selection of three Wrapped Instruments, while Figure 6 its hierarchical representation.

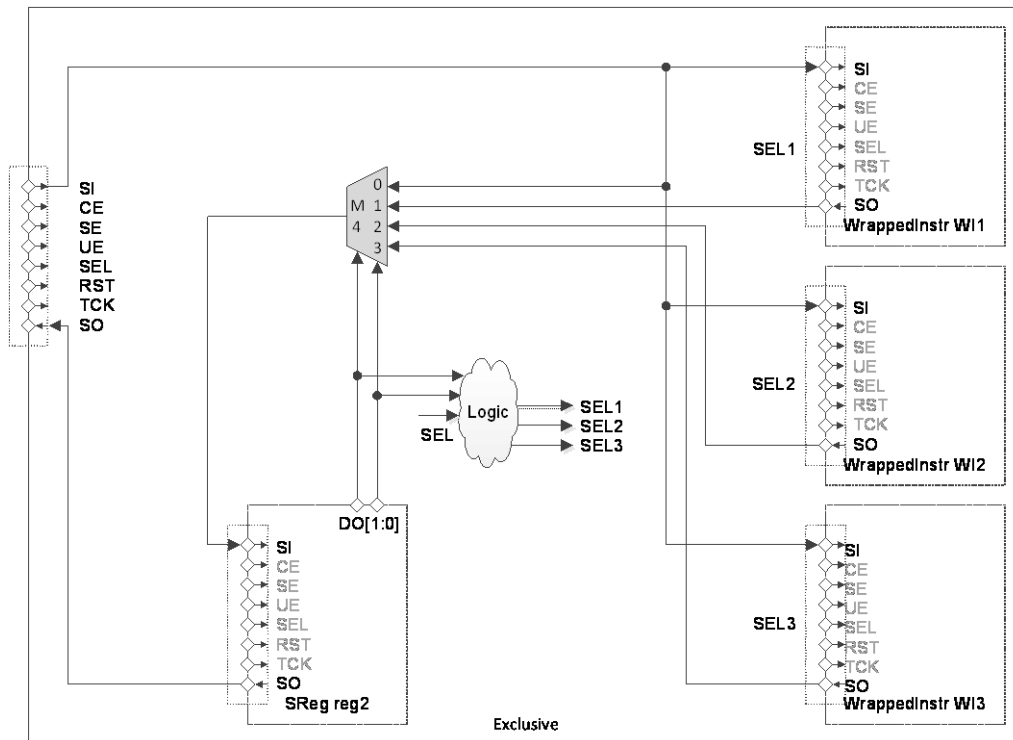


Figure 5 Example E-14 from the IEEE 1687-2014 standard document

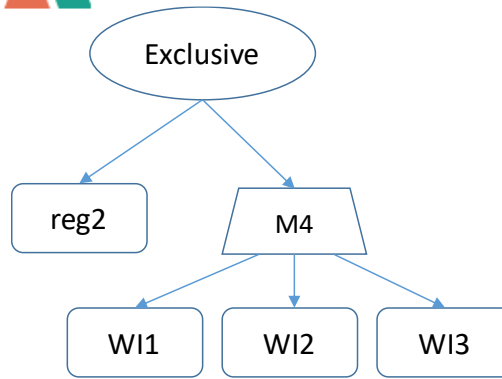


Figure 6 Hierarchical representation of Figure 5

As stated in Section 7, the “Linker” M4 is associated with a SelectionRegister (like in ICL) and PathSelector that decodes its value. The SIT default options are:

- Binary: each WI1 is associated with a binary number in ascending order: “0b01” for WI1, “0b10” for WI2 and “0b11” for WI3. With “0b00” M4 is closed.
- Binary_noidle: the same as for binary, but M4 cannot be completely closed, so one derivation is always selected. Coding starts from 0: “0b00” for WI1, “0b01” for WI2 and “0b10” for WI3.
- One_Hot: one-hot coding. “0b001” for WI1, “0b010” for WI2 and “0b100” for WI3. With “0b000” M4 is closed
- One_Hot_noidle : the same as per one-hot, but the linker cannot be completely closed, so at least one segment must be chosen.
- Table-Based: in this case, a table explicitly provides the selection and deselection tables. It is the most similar to ICL.

The predefined path selectors cover the most usual implementation of a scan mux, with the table-based that can be used for the more unorthodox setups. Thanks to this abstraction, SIT is able to represent not only Muxes, but also OneHotGroups in a simple and compact notation.

4 AccessInterfaces

ICL is only able to describe the structural network inside a 1687 network: connections to the outside are depicted by an “AccessLink”, a simple placeholder for an 1149.1 Test Access Port (TAP) or equivalent. The actual implementation of an AccessLink is up to the retargeter. As detailed in [2], MAST implements an abstraction called AccessInterface, which enables to describe and exploit interfaces alternative to JTAG. An AccessLink node defines a “protocol”, which defines the transactions needed to access its children. So, for instance, an SVF AccessInterface has two children, the IR and DR chains, and its protocol defined two transactions: “SIR” and “SDR” respectively, as depicted in Figure 7.

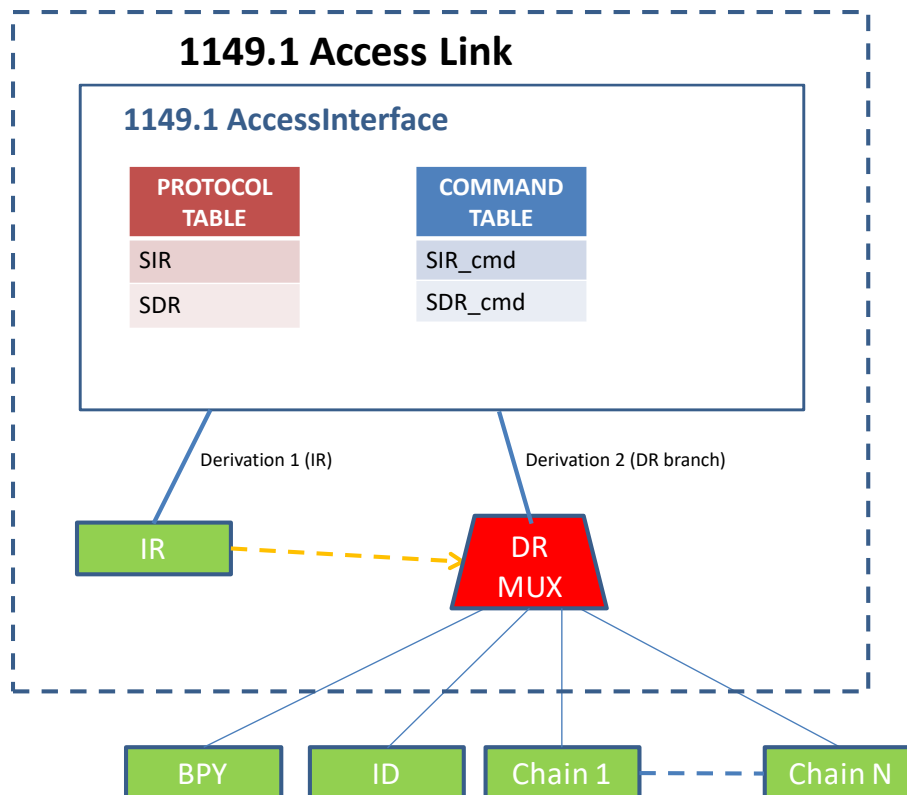


Figure 7 AccessInterface abstraction for an 1149.1 AccessLink, from [2]

This abstraction is implemented inside MAST, so in a SIT there is no need to explicitly describe the internal workings of an AccessInterface: all that is needed is to instantiate it with the correct Protocol. For instance, a complete 1149.1 TAP would look like this in SIT:

(1) ACCESS_INTERFACE TAP SVF_Emulation



(2) (

(3) REGISTER TAP_IR 4 Hold_value Bypass: "0b1111"

(4) LINKER TAP_DR_Mux Table_Based TAP_IR 2 NOT_IN_PATH

(5) (

(6) REGISTER TAP_BPY 1 Bypass: "0b1"

(7) REGISTER reg 12 Bypass: "0xABC"

(8))

(9))

In line (1), the AccessInterface is declared, specifying that the protocol callback is an “Emulation of SVF” (i.e. MAST will compute the SVF commands and write them in a log for future usage). The node can select two different path: the register TAP_IR and the linker TAP_DR_MUX. “SVF_Emulation” specifies how SIR and SDR commands must be formatted respectively.

As these constructs are quite common, SIT defines some macros for a more compact and efficient coding:

(1) JTAG_TAP TAP Loopback 4 1

(2) PDL Incr

(3) (

(4) REGISTER reg 12 Bypass: "0xABC"

(5))

Section 7 will provide more details on the AccessInterface instantiation syntax and its related macros.



5 Translators

Following P1687.1 proposals, SIT allows the definition of Translator interfaces, which use callbacks to transform operations from one protocol to the other. The syntax is the same as per `AccessInterface`. For instance, this code describes the same system as in Section 5 but in a more detailed way:

```
(1) TRANSLATOR top Emulation
(2) ACCESS_INTERFACE TAP SVF_Raw
(3) (
(4) REGISTER TAP_IR 4 Hold_value Bypass: "0b1111"
(5) LINKER TAP_DR_Mux Table_Based TAP_IR 2 NOT_IN_PATH
(6) (
(7) REGISTER TAP_BPY 1 Bypass: "0b1"
(8) REGISTER reg 12 Bypass: "0xABC"
(9) )
(10) )
(11) )
```

Translators can be declared hierarchically to describe a series of transformations, as in the following example where a JTAG-to-I2C translator is put between the TAP and the outside world.

```
(1) TRANSLATOR top Emulation
(2) (
(3) TRANSLATOR my_trans JTAG_to_I2C "0x40, 0x41, 0x42"
(4) (
(5) JTAG_TAP TAP 4 1
```



(6) PDL Console_Incr

(7) (

(8) REGISTER regHI 12 Bypass: "0xABC"

(9))

(10))

(11))



6 SIT support for custom modules

Thanks to its modularity, MAST can accept external modules to provide enhanced capabilities:

- Custom Path Selector for Linkers;
- Custom Protocols for AccessInterface;

This feature allows customer to implement their own proprietary hardware and still have MAST retarget through their design. This is implemented using a well-defined interface that is being proposed for standardization at the IEEE P1687.1 working group.



7 Complete SIT Syntax (from Doxygen) **OUTDATED, REFER TO REPOSITORY**

This module is based on flex/bison and provides an interface to generate a System Model from a simplified language, called Simplified [ICL](#) Tree ([SIT](#)).
The Syntax is extremely simple:

7.1 LEAF NODES

Leaf nodes are nodes that cannot have children because they represent the basic 1687 topology unit : the register.

Syntax is:

```
REGISTER <node_name> <Size> Hold_value* Bypass : "<bypass_value>"
```

Node Type	Node Information	
REGISTER	<Size> Hold_value* Bypass: "<bypass_value>"	
	<Size>	number of bits composing the register in decimal notation
	Hold_value	if provided, sets the "hold" property of the register to true (i.e. the bypass value corresponds to the last value it has been assigned)
	<Bypass_value>	the bypass sequence, written following MAST's BinaryVector convention

Another leaf node is 'INSTANCE' that represents the instantiation of a module at the point of insertion (as for any other child nodes).

Syntax is:

```
INSTANCE <name> OF <sit_file> | <factory_name>
```

Node Type	Node Information	
INSTANCE	<code><name> OF <sit_file></code> <code><name> OF <factory_name></code>	
	<code><name></code>	instance name
	<code><sit_file></code>	name or path of a SIT file with .sit extension must be enclosed between double quotes if it contains spaces or reserved keywords
	<code><factory_name></code>	identify a model factory registered by a plugin

7.2 Parent NODES

Parent nodes represent hierarchical information: they can be either 1687 elements, such as [ICL](#) instances, or abstract constructs to help representing hierarchy. Each node declaration follows the same syntax:

```

<PDL_association>* : PDL <function_name_list>
<node_type> <node_name> <node information>="">* <PDL_association>* { <children_list>+ }
<node_type> <node_name> <node information>="">* <PDL_association>* ( <children_list>+ )

```

<code><node_type></code> :	it can be either a base node (REGISTER,CHAIN,LINKER,ACCESS_INTERFACE) or a composite node identifying a specific configuration (ex: SIB , 1500_WRAPPER, AI_1149_1, TAP _1149_1, etc...).
<code><node_name></code>	standard identifier. Can be shared by different nodes (the Builder will assign an unique identifier)
<code><PDL_association></code>	used to indicate that the function identified by "function_name_list" (comma-separated identifiers) is to be associated with the current Parent Node (see main documentation for details). It is the equivalent of the PDL <code>iProcsForModule</code> command.
<code><children_list>+</code>	non-empty list of nodes in the lower hierarchical level
<code><node information>=""></code>	a variable field containing information specific to a given node_type, as explained in the following table. NB: underlined strings depict language



token

There are two types of possible hierarchies: chained or linked.

7.2.1 CHAINED hierarchy

In a chained hierarchy, all children nodes are always selected from left to right. As its name says, this rules models a scan chain composed of multiple segments, and it also the easiest way to add hierarchical information to a design. Chained hierarchies are identified by Accolades.

The children of a given node are listed between two accolades following it.
The order in the list is respected when appending children.
For instance:

```
CHAIN test_chain
{
  REGISTER test_register_1 12 Bypass: "0b1001:0110:1100"
  REGISTER test_register_2 12 Bypass: "0b1001:0110:1100"
}
```

Register nodes test_register_1 and test_register_2 are respectively the first and second child of chain node test_chain.

Due to Lex/Yacc limitations, SIT_Reader does not support empty hierarchies. At least one node must be provided.

NB: Trying to provide children to a Register node will cause a syntax error.

7.2.1.1 NODE INFORMATION DEPENDING ON NODE TYPE FOR CHAINED HIERARCHIES

Node Type	Node Information
CHAIN	NOT_IN_PATH*: if provided, the name of the chain is not used when composing a path (used by PDL to identify registers)

7.2.2 LINKED hierarchy

In a selected hierarchy, not all children are necessary active at the same time: it depends on the internal state of the System, typically expressed, as its names says, by a Linker node. Linker hierarchies also comprise AccessInterfaces and the macro derived from one of more Linkers, like SIBs or MIBs. Linked Hierarchies are identified by Parenthesis



The children of a given node are listed between two parenthesis following it.
The order in the list is respected when applying the selection protocol
For instance:

```
LINKER test_LINKER One_Hot <selector> 2
(
  REGISTER test_register_1 12 Bypass: "0b1001:0110:1100"
  REGISTER test_register_2 12 Bypass: "0b1001:0110:1100"
)
```

The following table identifies the possible syntaxes for <selector>, which can be freely mixed.

Syntax	Meaning
<name>	The register identified by the (unique) name is used as the selector for the Linker
<name>[range]	The slice of register identified by the (unique) name and the range is used as the selector for the Linker. Range can either be a single bit (ex: name[0]) or and interval (ex: name [0:2])
{<parent>}+.<name>	Same as per <name>, but <parent> can be used to specify the hierarchy where "name" is defined to avoid ambiguity. For instance: A.B.reg or C.F.G.reg
<selector>{:<selector>}+	<selector>s are concatenated to obtain the Virtual Register selecting the linker. For instance reg1:reg2 or regA[0]:a.b.regB[0:3]

nodes test_register_1 and test_register_2 are respectively the first and second child of linker node test_linker.

Due to Lex/Yacc limitations, SIT_Reader does not support empty hierarchies. At least one node must be provided.

NB: Trying to provide children to a Register node will cause a syntax error.

7.2.2.1 NODE INFORMATION DEPENDING ON NODE TYPE FOR LINKED HIERARCHIES

Node Type	Node Information
MIB	Macro to generate a n-bit register controlling a n- derivation linker. <position> <active> <reverse>* <max_derivations> <path_selector>

	<position>	can be either PRE (the control register precedes the Linker) or POST (the control register follows the Linker)
	<active>	can be either HIGH (active-high logic selection) or LOW (active-low logic selection)
	<reverse>	if set to REVERSE, bit mapping in the control register is reversed (i.e. leftmost bit selects rightmost derivation)
	<max_derivations>	the maximum number of derivations accepted by the MIB, as a decimal number. NB: a MIB can have fewer derivations that its maximum
	<path_selector>	a string selecting the decoding strategy. See "Linker" for this list and description of supported selectors
SIB	Macro to generate a 1-bit register controlling a 1- derivation linker, as defined in the standard document. <position> <active>	
	<position>	can be either PRE (the control register precedes the Linker) or POST (the control register follows the Linker)
	<active>	can be either HIGH (active-high logic selection) or LOW (active-low logic selection)
WRAPPER_1500	IEEE-1500 compliant wrapper derivations. Selector is set as Binary	
	<max_derivations>	Maximum number of wrapper registers
LINKER	<path_selector> <ctrl_reg> <max_derivations>	
	<path_selector>	a string selecting the decoding strategy of the Linker

	<ctrl_reg>	string denoting the "node_name" of the register controlling the linker
	<max_derivations>	maximum number of allowed paths (i.e. number of children)
	NB: the control register can be declared anywhere in the SIT file (not necessarily before the Linker).	
	Binary	Derivation "n" is selected by putting the binary value "n" in the controlling register. When set to 0, no path is selected. Selection register size is $\text{ceiling}(\log_2(\text{path_count}+1))$
	Binary_noidle	Derivation "n" is selected by putting the binary value "n-1" in the controlling register. When put to 0, path 1 is selected. Selection register size is $\text{ceiling}(\log_2(\text{path_count}))$
	One_Hot	Derivation "n" is selected by putting the n-th register of the controlling path to 1. When set to 0, no path is selected. Only one path can be opened at a given time. Selection register size is "path_count"
	One_Hot_noidle	Like One_Hot, but a path is always be selected (i.e. there is no "no path selected" coding)
	N_Hot	Derivation "n" is selected by putting the n-th register of the controlling path to 1. When set to 0, no path is selected. Several paths can be opened at a given time. Selection register size is "path_count"
	N_Hot_noidle	Like N_Hot, but at least one path must always be

		selected (i.e. there is no "no path selected" coding) Table_Based Not supported yet
ACCESS_INTERFACE	<protocol> <AI_identifier>* <AI_table> Creates an Access Interface node implementing the provided protocol	
	<protocol>	the protocol used by the underlying AccessInterface. Can be JTAG_Loopback, JTAG_SVF_simulation, JTAG_SVF_Emulation or SPI_FTDI
	<AI_identifier>	an optional string that might be required by the protocol. For instance, SPI_FTDI might specify an usbDeviceID as quoted-string bit/hex value.
	<AI_TABLE>	the addresses used by the protocol provided as a comma-separated list between square brackets. Parenthesis can be used for visual help but have no semantic value. For instance: [("0x41" , "0x42" ,"0x0") , ("0x1" , "0x2" ,"0x3")] == ["0x41" , "0x42" ,"0x0" , "0x1" , "0x2" ,"0x3"] == [("0x41" , "0x42"),"0x0" , ("0x1" , "0x2" ,"0x3"]
	NB: openOCD protocol supposes a fully-formed TAP in the System Under Test, so it can be instantiated only inside the TAP macro	
JTAG_TAP	<protocol> <optional_AI_identifier>* <IR_size> <optional_IR_coding>* <max_DR_chains> Creates an IEEE-1149.1 compliant Test Access Port composed by a AI_1149_1 node having a "IR_size" IR register controlling a DR_MUX for "chain count" endpoints, with a 1-bit bypass register already appended. When no coding is provided, BYPASS is selected by FFFF (all ones), and chains by a binary selection starting at 0x01.	
	<protocol>	the protocol used by the underlying

		AccessInterface. Can be Loopback, SVF_simulation or SVF_openOCD
	<optional_AI_identifier>	an optional string that might be required by the protocol. For instance, SVF_openOCD requires a Design Name to be provided
	<IR_size>	decimal number indicating the size in bits of the IR register
	<max_DR_chains>	decimal number indicating the maximum number of chains
	<optional_IR_coding>	the coding used by the IR, starting from the Bypass Register, provided as a comma-separated list between square brackets. Parenthesis can be used for visual help but have no semantic value. For instance, ["0xFF" , "0x42" ,"0x00"] == ["0xFF" , ("0x42" ,"0x00")] == [("0xFF" , "0x42" ,("0x00")

7.3 USAGE

To use the SIT_reader, the [SIT Reader.hpp](#) header must be included.

A reader object can be instantiated by giving it a `std::shared_ptr<mast::SystemModel>` to work on as parameter, for instance:

```
SIT::SIT_Reader driver(sm);
```

Parsing can then be launched using the "parse" method, with either a filename or an istream as input:

```
auto result = driver.parse(stream);
```

If parsing is correctly executed, the return value is **true** and `driver.parsed_sut` points to the root of the generated tree.

If an error occurs, the return value is **false** and `driver.parsed_sut == nullptr`.



NB: [SIT](#) parsing is not supposed to raise any exception or cause execution to stop, but the SystemModel build process raises exceptions as usual.

8 Examples from the IEEE 1687-2014 Standard Document

In this section, we will analyse the ICL examples given in the standard document and provide their SIT counterparts. All figures in the section are @ IEEE.

8.1 Annex E, Example 1

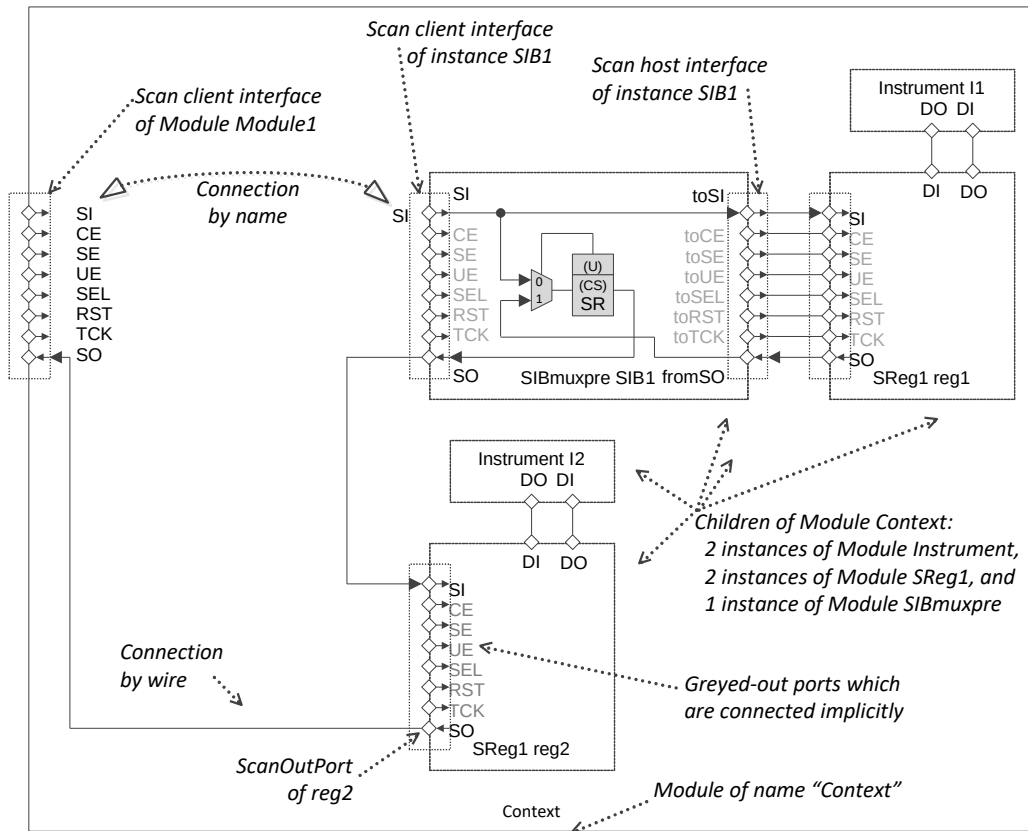


Figure E.1— Context for subsequent examples

ICL :

```
Module Context {
    ScanInPort    SI;

    ScanOutPort   SO { Source reg2.SO; }

    ShiftEnPort   SE;

    CaptureEnPort CE;
```



SelectPort SEL;

ResetPort RST;

TCKPort TCK;

```
Instance SIB1 Of SIB_mux_pre {
    InputPort SI = SI;
    InputPort fromSO = reg1.SO;
}

Instance I1 Of Instrument { InputPort DI = reg1.DO; }

Instance reg1 Of SReg {
    InputPort SI = SIB1.toSI;
    InputPort DI = I1.DO;
}

Instance I2 Of Instrument { InputPort DI = reg2.DO; }

Instance reg2 Of SReg {
    InputPort SI = SIB1.SO;
    InputPort DI = I2.DO;
}
}
```

SIT :

```
Chain Context {
    SIB SIB1 PRE HIGH
    ( Instance reg1 Of SReg )
    Instance reg2 Of SReg
}
```

Apart from the much more compact description, in the SIT file the hierarchical relation between reg1 and SIB1 is explicitly stated, making it much more readable.

8.2 Annex E, Example 2

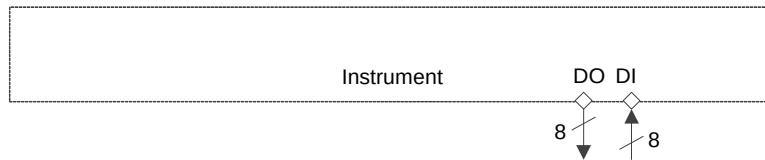


Figure E.2—Example generic instrument (8 bits wide)

As explained in Section 2.3, SIT does not need explicit provisions for Raw instruments, but PDL can still be used exactly as in the example.

8.3 Annex E, Example 3

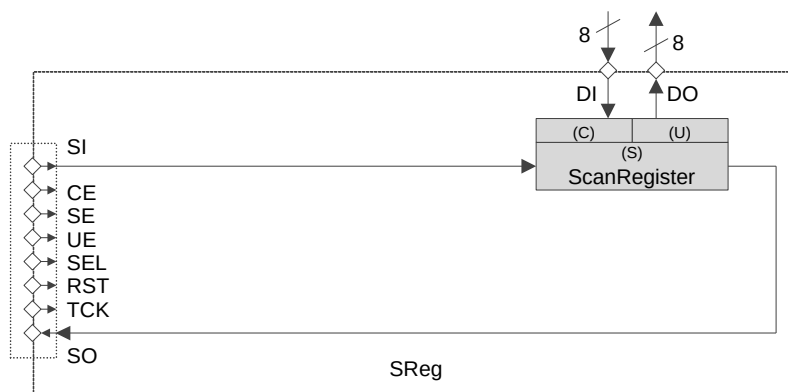


Figure E.3—Scan register example (8 bits)

ICL :

```
Module SReg {
  ScanInPort    SI;
  ScanOutPort   SO { Source SR[0]; }
  ShiftEnPort   SE;
  CaptureEnPort CE;
  UpdateEnPort  UE;
  SelectPort    SEL;
  ResetPort     RST;
  TCKPort       TCK;
  DataInPort    DI[7:0];
  DataOutPort   DO[7:0] {Source SR; }
  ScanInterface scan_client { Port SI; Port SO; Port SEL; }

  ScanRegister SR[7:0] { ScanInSource SI;
                        CaptureSource DI;
                        ResetValue 8'b00000000; }
}
```



SIT :

REGISTER SR 8 Bypass: "0x00"

Parametric ICL :

```
Module SReg {
  Parameter MSB = 7;
  ScanInPort    SI;
  ScanOutPort   SO { Source  SR[0];}
  ShiftEnPort   SE;
  CaptureEnPort CE;
  UpdateEnPort  UE;
  SelectPort    SEL;
  ResetPort     RST;
  TCKPort       TCK;
  DataInPort    DI[$MSB:0];
  DataOutPort   DO[$MSB:0] {Source SR; }
  ScanInterface scan_client { Port SI; Port SO; Port SEL; }

  ScanRegister SR[$MSB:0] { ScanInSource SI;
                           CaptureSource DI;
                           ResetValue 'b0; }
}
```

Parametric SIT :

<Not implemented yet>

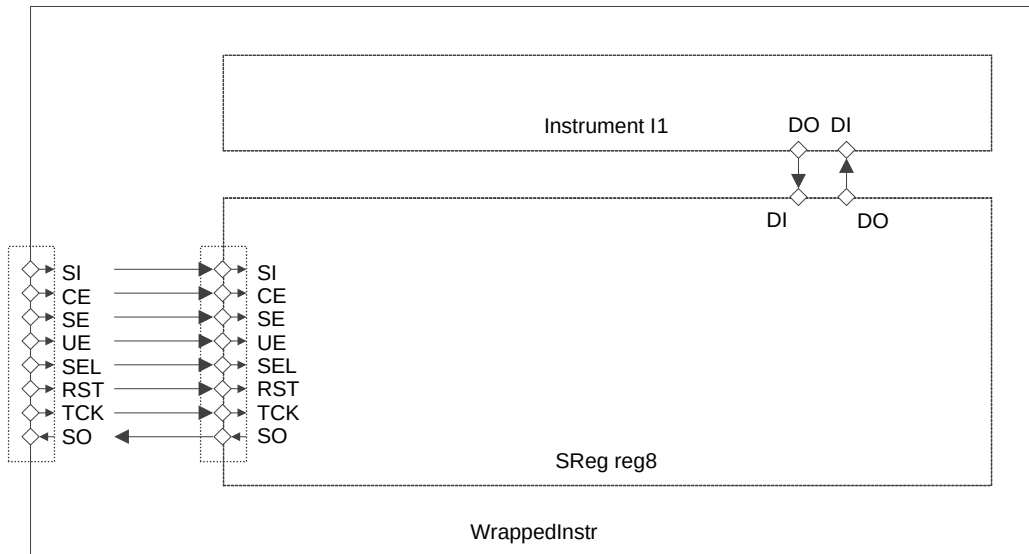


Figure 8.4—Wrapped instrument example

ICL :

```
Module WrappedInstr {
  ScanInPort    SI;
  ScanOutPort   SO { Source reg8.SO; }
  ShiftEnPort   SE;
  CaptureEnPort CE;
  UpdateEnPort  UE;
  SelectPort    SEL;
  ResetPort     RST;
  TCKPort       TCK;
  ScanInterface scan_client { Port SI; Port SO; Port SEL; }

  Instance I1 Of Instrument { InputPort DI = reg8.DO; }
  Instance reg8 Of SReg {
    InputPort SI = SI; InputPort DI = I1.DO; Parameter Size = 8;
  }
}
```

SIT :

```
CHAIN WrappedInstr
{INSTANCE reg8 of SReg.sit}
```

NB : in SIT terms, I1 is implicit

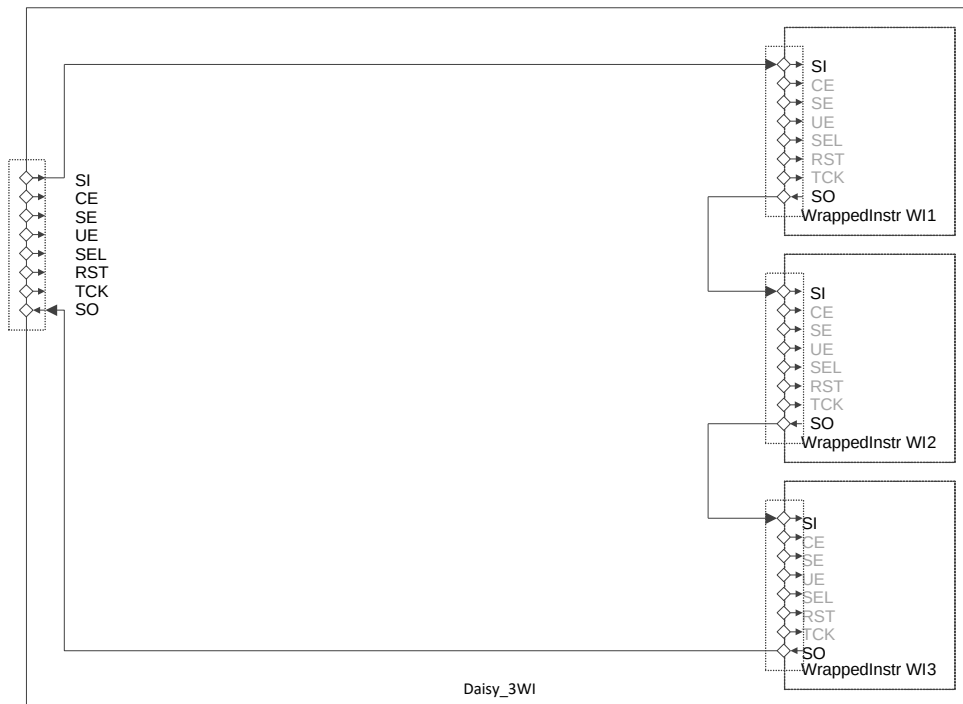


Figure 8.5—Daisy Chain example

ICL:

```
Module Daisy_3WI {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source WI3.SO; }
  Instance WI1 Of WrappedInstr { InputPort SI = SI; }
  Instance WI2 Of WrappedInstr { InputPort SI = WI1.SO; }
  Instance WI3 Of WrappedInstr { InputPort SI = WI2.SO; }
}
```

SIT:

```
CHAIN Daisy_3WI
{
  Instance WI1 Of WrappedInstr
  Instance WI2 Of WrappedInstr
  Instance WI3 Of WrappedInstr
}
```

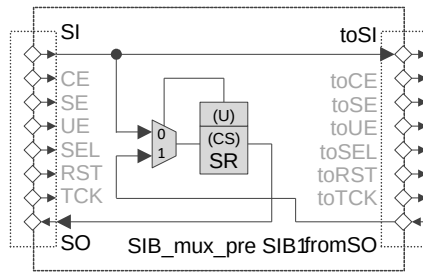


Figure 8.6—SIB (mux pre reg) example

ICL :

```
Module SIB_mux_pre {
  ScanInPort      SI;
  CaptureEnPort   CE;
  ShiftEnPort     SE;
  UpdateEnPort    UE;
  SelectPort      SEL;
  ResetPort       RST;
  TCKPort         TCK;
  ScanOutPort     SO { Source SR; }
  ScanInterface client {
    Port SI; Port CE; Port SE; Port UE;
    Port SEL; Port RST; Port TCK; Port SO;
  }
  ScanInPort      fromSO;
  ToCaptureEnPort toCE;
  ToShiftEnPort   toSE;
  ToUpdateEnPort  toUE;
  ToSelectPort    toSEL;
  ToResetPort     toRST;
  ToTCKPort       toTCK;
  ScanOutPort     toSI { Source SI; }
  ScanInterface host {
    Port fromSO; Port toCE; Port toSE; Port toUE;
    Port toSEL; Port toRST; Port toTCK; Port toSI;
  }
  ScanRegister SR {
    ScanInSource SIBmux; CaptureSource SR; ResetValue 1'b0;
  }
  ScanMux SIBmux SelectedBy SR {
    1'b0 : SI;
    1'b1 : fromSO;
  }
}
```

SIT (explicit):

```
CHAIN SIB_mux_pre
{
  LINKER Mux Binary SR 1
  ( )
  REGISTER SR 1 Bypass: "0x0"
```



SIT (compact):

SIB sib POST HIGH

(...)

8.7 Annex E, Example 7

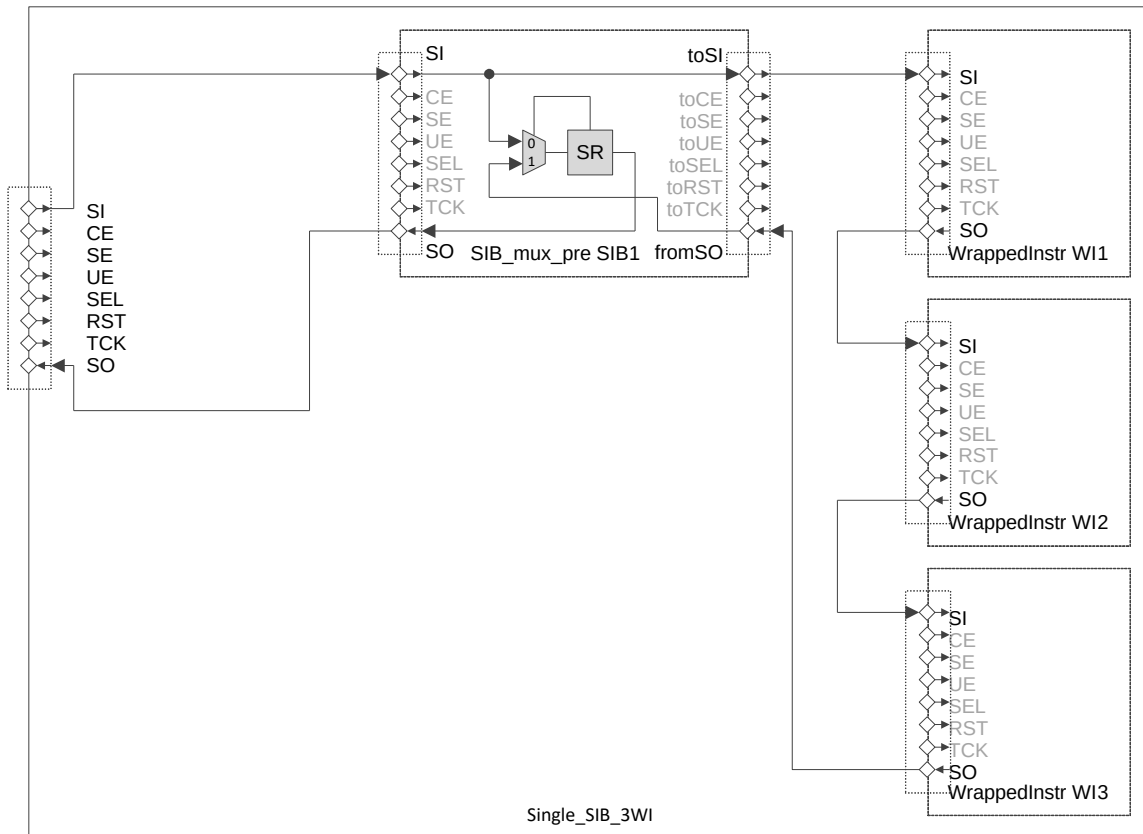


Figure 8.7—serial chain of 3 wrapped instruments behind a single SIB

ICL :

```
Module Single_SIB_3WI {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source SIB1.SO; }

  Instance SIB1 Of SIB_mux_pre { InputPort SI = SI;
                                InputPort fromSO = WI3.SO; }
  Instance WI1 Of WrappedInstr { InputPort SI = SIB1.toSI; }
  Instance WI2 Of WrappedInstr { InputPort SI = WI1.SO; }
  Instance WI3 Of WrappedInstr { InputPort SI = WI2.SO; }
}
```

SIT (explicit):



```
CHAIN SIB_mux_pre
{
  LINKER Mux Binary SR 1
  (
    {
      Instance WI1 Of WrappedInstr
      Instance WI2 Of WrappedInstr
      Instance WI3 Of WrappedInstr
    }
  )
  REGISTER SR 1 Bypass: "0x0"
}
```

SIT (compact):

```
SIB sib POST HIGH
(
  {
    Instance WI1 Of WrappedInstr
    Instance WI2 Of WrappedInstr
    Instance WI3 Of WrappedInstr
  }
)
```

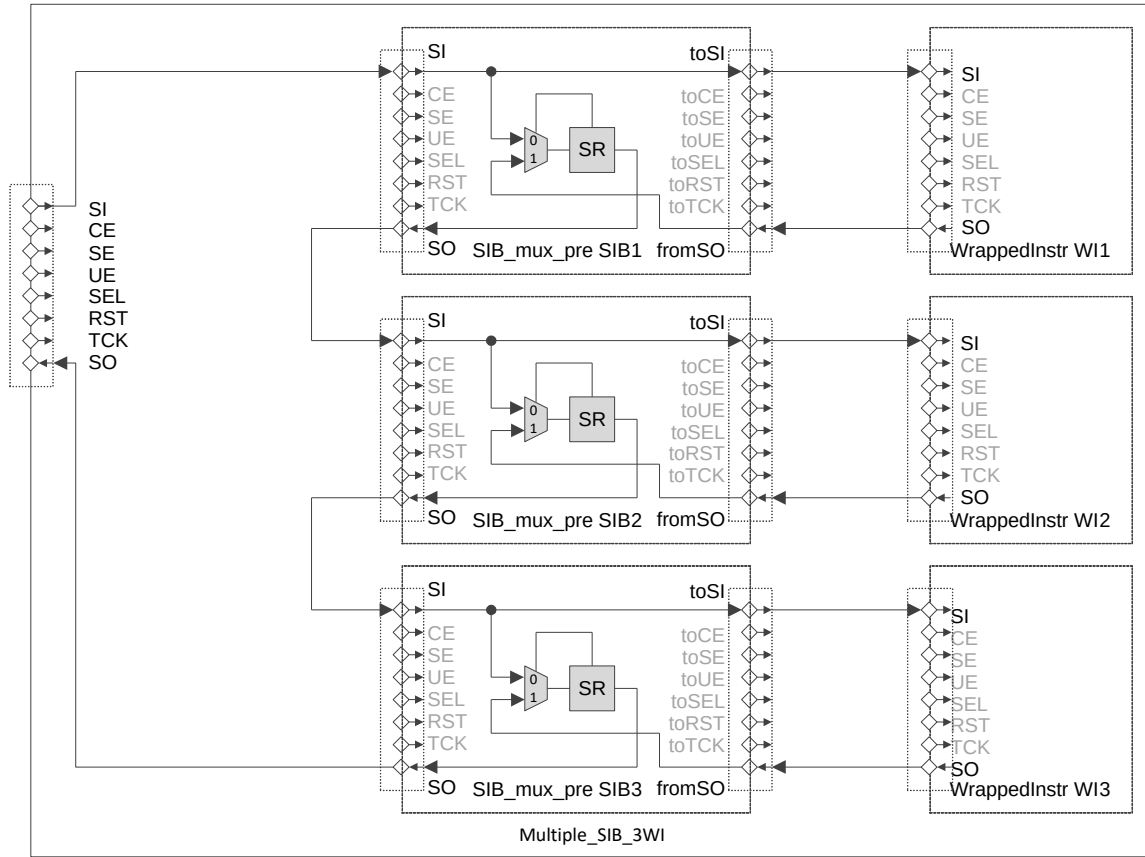


Figure 8.8—3 wrapped instruments, each behind its own SIB

ICL :

```
Module Multiple_SIB_3WI {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source SIB3.SO; }

  Instance SIB1 Of SIB_mux_pre { InputPort SI = SI;
                                InputPort fromSO = WI1.SO; }
  Instance SIB2 Of SIB_mux_pre { InputPort SI = SIB1.SO;
                                InputPort fromSO = WI2.SO; }
  Instance SIB3 Of SIB_mux_pre { InputPort SI = SIB2.SO;
                                InputPort fromSO = WI3.SO; }
  Instance WI1 Of WrappedInstr { InputPort SI = SIB1.toSI; }
  Instance WI2 Of WrappedInstr { InputPort SI = SIB2.toSI; }
  Instance WI3 Of WrappedInstr { InputPort SI = SIB3.toSI; }
}
```

SIT (compact):

```
CHAIN Multiple_SIB_3WI
```



```

{
  SIB SIB1 POST HIGH
    ( Instance WI1 Of WrappedInstr )
  SIB SIB2 POST HIGH
    ( Instance WI2 Of WrappedInstr )
  SIB SIB3 POST HIGH
    ( Instance WI3 Of WrappedInstr )
}

```

8.9 Annex E, Example 9

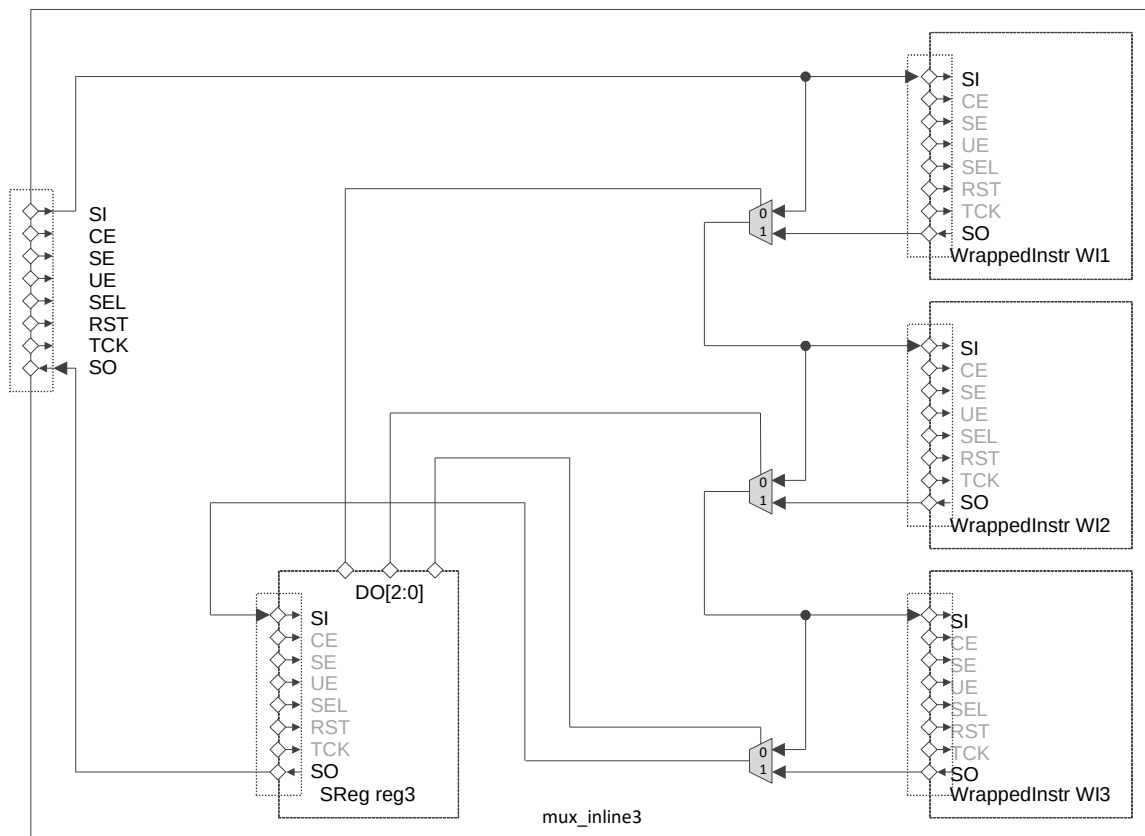


Figure 8.9— Scan muxes with local control

ICL:

```

Module mux_inline3 {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source reg3.SO; }

  Instance WI1 Of WrappedInstr { InputPort SI = SI; }

```



```
Instance WI2 Of WrappedInstr { InputPort SI = mux1; }
Instance WI3 Of WrappedInstr { InputPort SI = mux2; }
Instance reg3 Of SReg { InputPort SI = mux3; Parameter Size = 3;
    InputPort DI = 'b0; }
ScanMux mux1 SelectedBy reg3.DO[2] {
    1'b0 : SI;
    1'b1 : WI1.SO;
}
ScanMux mux2 SelectedBy reg3.DO[1] {
    1'b0 : mux1;
    1'b1 : WI2.SO;
}
ScanMux mux3 SelectedBy reg3.DO[0] {
    1'b0 : mux2;
    1'b1 : WI3.SO;
}
}
```

SIT:

```
CHAIN mux_inline3
{
    LINKER Mux1 Binary Reg3.SR[2] 1
    ( Instance WI1 Of WrappedInstr )
    LINKER Mux2 Binary Reg3.SR[1] 1
    ( Instance WI2 Of WrappedInstr )
    LINKER Mux3 Binary Reg3.SR[0] 1
    ( Instance WI3 Of WrappedInstr )
    INSTANCE Reg3 of SReg.sit REGISTER SR 3 Bypass: "0x0"
}
```

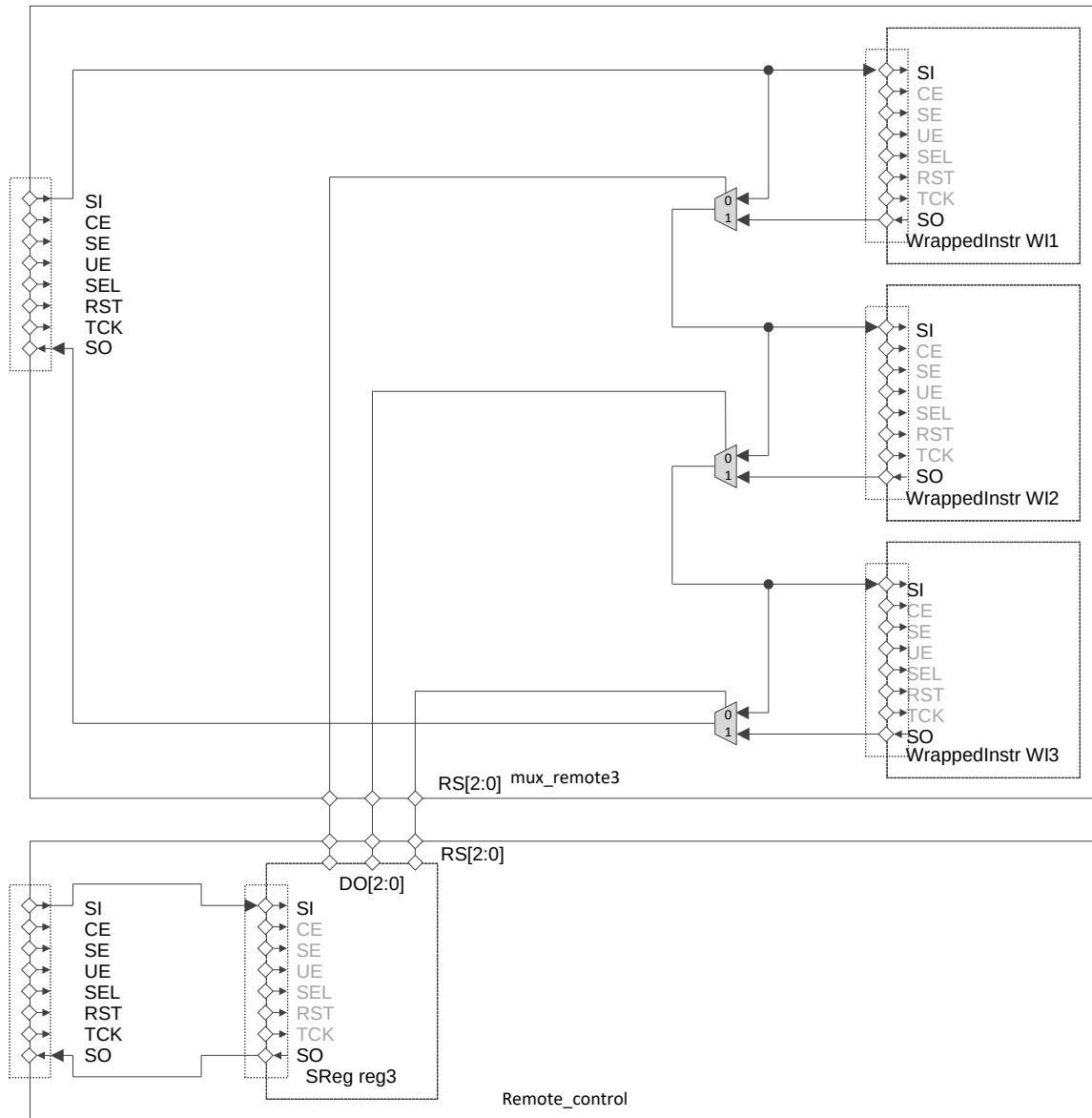


Figure 8.10—Scan muxes with remote control

ICL :

```
Module mux_remote3 {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source mux3; } DataInPort RS[2:0];

  Instance WI1 Of WrappedInstr { InputPort SI = SI; }
  Instance WI2 Of WrappedInstr { InputPort SI = mux1; }
  Instance WI3 Of WrappedInstr { InputPort SI = mux2; }
```



```
ScanMux mux1 SelectedBy RS[2] {
  1'b0 : SI;
  1'b1 : WI1.SO;
}
ScanMux mux2 SelectedBy RS[1] {
  1'b0 : mux1;
  1'b1 : WI2.SO;
}
ScanMux mux3 SelectedBy RS[0] {
  1'b0 : mux2;
  1'b1 : WI3.SO;
}
}
Module remote_control {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source reg3.SO; }
  DataOutPort RS[2:0] { Source reg3.DO[2:0]; }

  Instance reg3 Of SReg { InputPort SI = SI; Parameter Size = 3;}
}
```

SIT:

```
CHAIN mux_remote3
{
  LINKER Mux1 Binary Reg3.SR[2] 1
  ( Instance WI1 Of WrappedInstr )
  LINKER Mux2 Binary Reg3.SR[1] 1
  ( Instance WI2 Of WrappedInstr )
  LINKER Mux3 Binary Reg3.SR[0] 1
  ( Instance WI3 Of WrappedInstr )
}
[.....]
CHAIN remote_control
{
  INSTANCE Reg3 of SReg.sit REGISTER SR 3 Bypass: "0x0"
}
```

<Not supported yet, need Aliases. But remote IS supported>

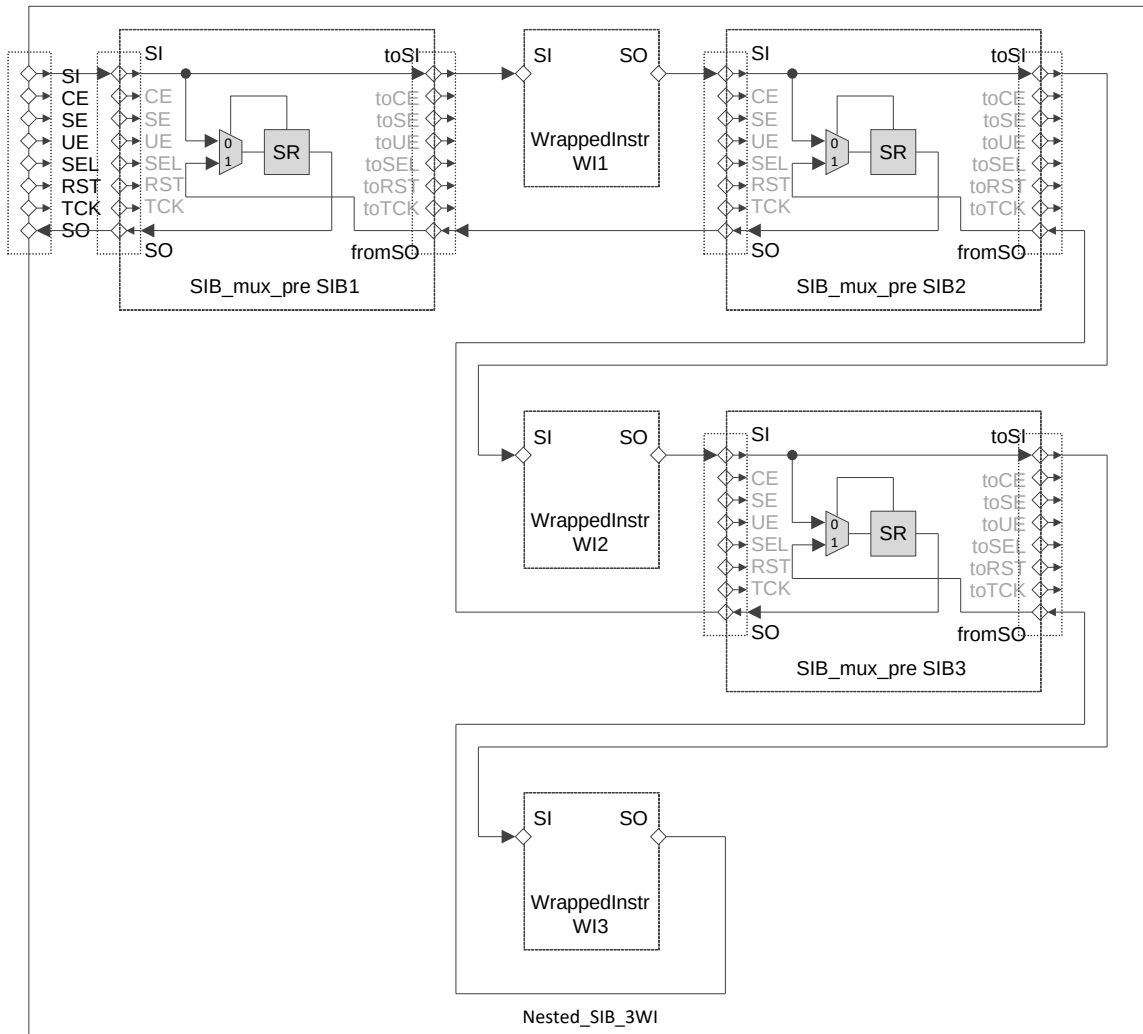


Figure 8.11—Nested SIBs: mux_pre

ICL :

```
Module Nested_SIB_3WI {
    ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
    UpdateEnPort UE; SelectPort SEL; ResetPort RST;
    TCKPort TCK; ScanOutPort SO { Source SIB3.SO; }

    Instance SIB1 Of SIB_mux_pre { InputPort SI = SI;
                                     InputPort fromSO = SIB2.SO; }
```



```
Instance SIB2 Of SIB_mux_pre { InputPort SI = WI1.SO;
                                InputPort fromSO = SIB3.SO; }

Instance SIB3 Of SIB_mux_pre { InputPort SI = WI2.SO;
                                InputPort fromSO = WI3.SO; }

Instance WI1 Of WrappedInstr { InputPort SI = SIB1.toSI; }
Instance WI2 Of WrappedInstr { InputPort SI = SIB2.toSI; }
Instance WI3 Of WrappedInstr { InputPort SI = SIB3.toSI; }

}
```

SIT (compact):

```
CHAIN Nested_SIB_3WI
{
  SIB SIB1 POST HIGH
  (
    {
      Instance WI1 Of WrappedInstr
      SIB SIB2 POST HIGH
      (
        {
          Instance WI2 Of WrappedInstr
          SIB SIB3 POST HIGH
          (
            Instance WI3 Of WrappedInstr
          )
        }
      )
    }
  )
}
```



```
Module BAD_Nested_SIB_3WI {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source SIB3.SO; }

  Instance SIB1 Of SIB_mux_post { InputPort SI = SI;
                                   InputPort fromSO = SIB2.SO; }
  Instance SIB2 Of SIB_mux_post { InputPort SI = WI1.SO;
                                   InputPort fromSO = SIB3.SO; }
  Instance SIB3 Of SIB_mux_post { InputPort SI = WI2.SO;
                                   InputPort fromSO = WI3.SO; }
  Instance WI1 Of WrappedInstr { InputPort SI = SIB1.toSI; }
  Instance WI2 Of WrappedInstr { InputPort SI = SIB2.toSI; }
  Instance WI3 Of WrappedInstr { InputPort SI = SIB3.toSI; }
}
```



SIT:

```
CHAIN BAD_Nested_SIB_3WI
{
  SIB SIB1 PRE HIGH
  (
    {
      Instance WI1 Of WrappedInstr
      SIB SIB2 PRE HIGH
      (
        {
          Instance WI2 Of WrappedInstr
          SIB SIB3 PRE HIGH
          (
            Instance WI3 Of WrappedInstr
          )
        }
      )
    }
  )
}
```

8.13 Annex E, Example 13

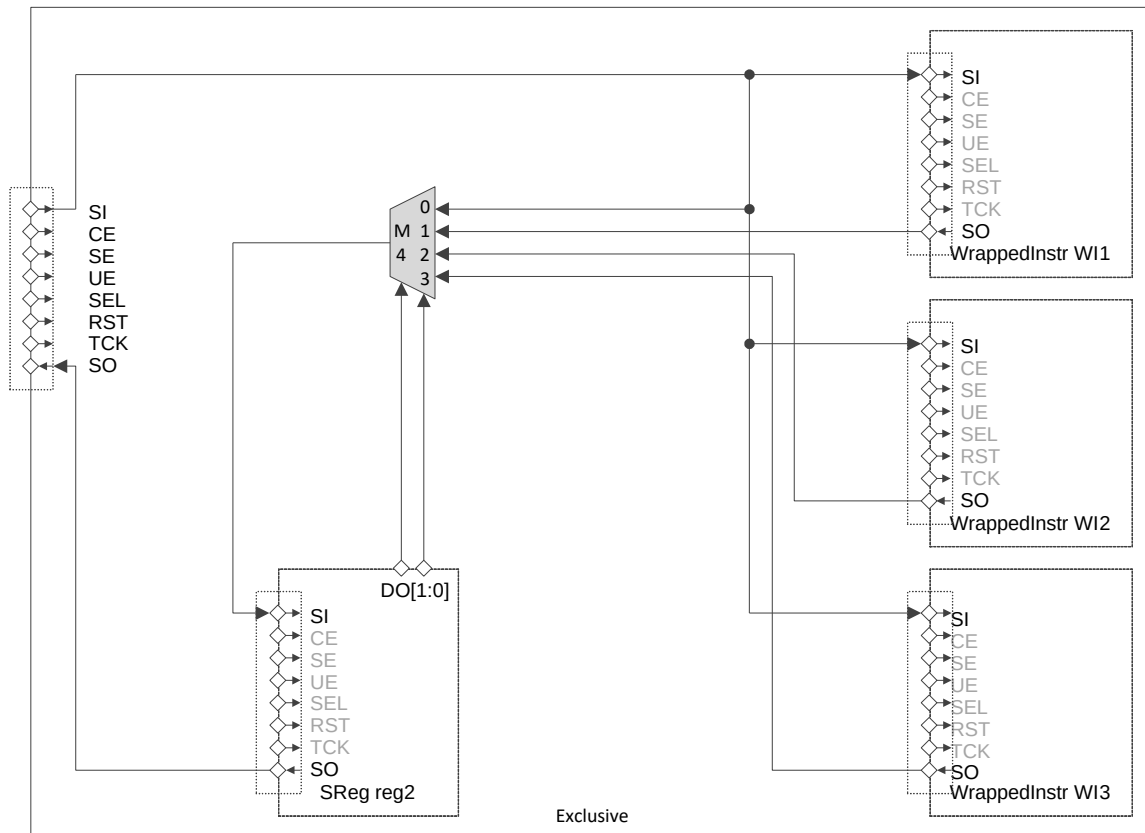


Figure 8.13—Exclusive access: implicit ICL



ICL:

```
Module Exclusive {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source reg2[0]; }

  Instance WI1 Of WrappedInstr { InputPort SI = SI; }
  Instance WI2 Of WrappedInstr { InputPort SI = SI; }
  Instance WI3 Of WrappedInstr { InputPort SI = SI; }

  ScanMux M4 SelectedBy reg2 {2'b00 : SI;
                              2'b01 : WI1.SO;
                              2'b10 : WI2.SO;
                              2'b11 : WI3.SO;
  }
  ScanRegister reg2[1:0] {ScanInSource M4;
                          CaptureSource 2'b00;
                          ResetValue 2'b00;
  }
}
```

SIT: The “Binary” path selector (Section 3) can be directly used

```
CHAIN Exclusive Binary
{
  REGISTER Reg2 2 Bypass: "0b00"
  LINKER Mux Binary Reg2 4
  (
    Instance WI1 Of WrappedInstr { InputPort SI = SI; }
    Instance WI2 Of WrappedInstr { InputPort SI = SI; }
    Instance WI3 Of WrappedInstr { InputPort SI = SI; }
  )
}
```

SIT (compact): SIT provides a “MIB” macro for easy instantiation (NB: Reg2 size is inferred as $\text{ceiling}(\log_2(\text{MIB_size}))=2$ in this case)

```
MIB Exclusive POST HIGH 3 Binary
(
  Instance WI1 Of WrappedInstr { InputPort SI = SI; }
  Instance WI2 Of WrappedInstr { InputPort SI = SI; }
  Instance WI3 Of WrappedInstr { InputPort SI = SI; }
)
```

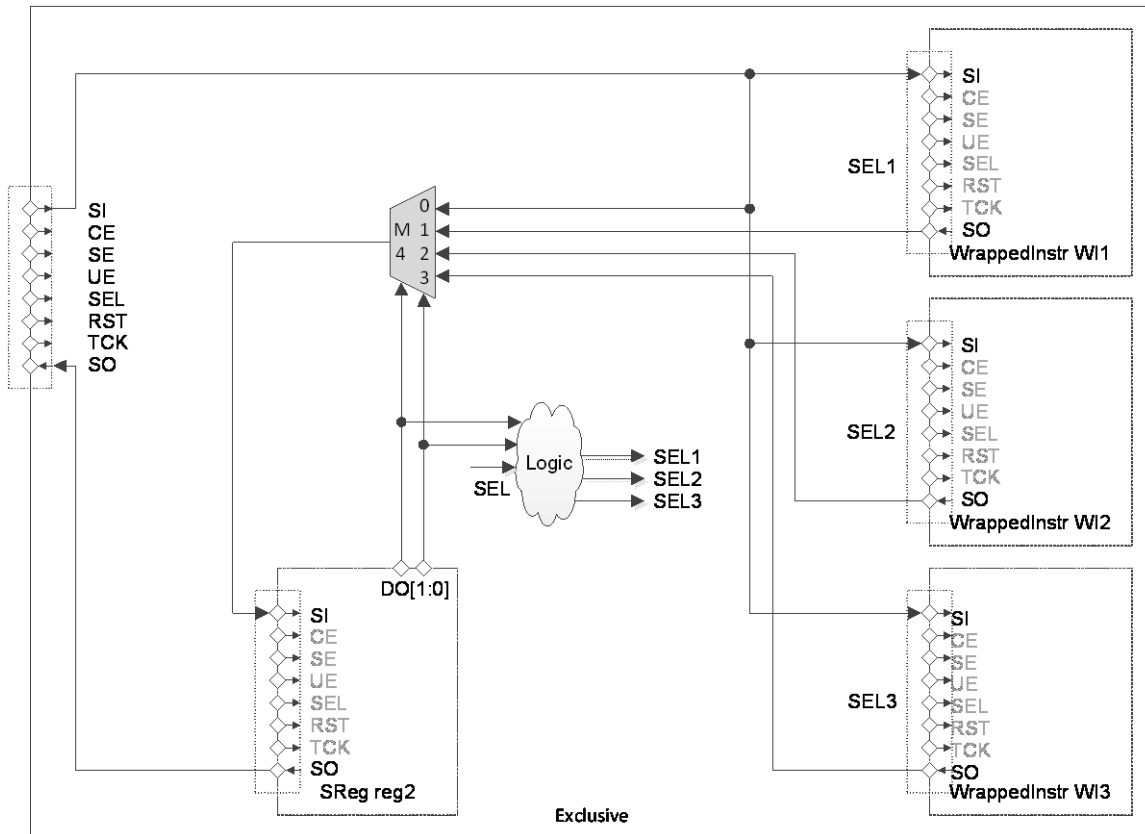


Figure 8.14—Exclusive access: explicit ICL

ICL:

```
Module Exclusive {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source reg2[0]; }

  Instance WI1 Of WrappedInstr {
    InputPort SI = SI;
    InputPort SEL = SEL1;
  }
  Instance WI2 Of WrappedInstr {
    InputPort SI = SI;
    InputPort SEL = SEL2;
  }
  Instance WI3 Of WrappedInstr {
    InputPort SI = SI;
    InputPort SEL = SEL3;
  }
  // This logic matches the SO selection logic of the mux
  LogicSignal SEL1 {(SEL,reg2[1],reg2[0]) == 3'b101;}
  LogicSignal SEL2 {(SEL,reg2[1],reg2[0]) == 3'b110;}
  LogicSignal SEL3 {(SEL,reg2[1],reg2[0]) == 3'b111;}
}
```



```
ScanMux M4 SelectedBy reg2 {2'b00 : SI;
                           2'b01 : WI1.SO;
                           2'b10 : WI2.SO;
                           2'b11 : WI3.SO;
                           }
ScanRegister reg2[1:0] {ScanInSource M4;
                       CaptureSource 2'b00;
                       ResetValue 2'b00;
                       }
}
```

SIT:

<This is done using a Table-Based Path Selector>

8.15 Annex E, Example 15

<MAST does not support broadcast as it is focused on single instrument access>

8.16 Annex E, Example 16

<MAST does not support broadcast as it is focused on single instrument access>

8.17 Annex E, Example 17

<MAST does not support broadcast as it is focused on single instrument access>

8.18 Annex E, Example 18

<MAST does not support broadcast as it is focused on single instrument access>



Figure 8.15—IEEE 1500 WSP

ICL :

```

Module WSP1500 {
  ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
  UpdateEnPort UE; SelectPort SEL; ResetPort RST;
  TCKPort TCK; ScanOutPort SO { Source IR_MUX; }
  DataInPort SWIR;

  ScanRegister WIR[1:0] {ScanInSource SI; ResetValue 2'b0;
                        CaptureSource 2'b01;};
  ScanRegister WBY      {ScanInSource SI; CaptureSource 1'b0;};

  Instance WI1 Of WrappedInstr { InputPort SI = SI; }
  Instance WI2 Of WrappedInstr { InputPort SI = SI; }
  Instance WI3 Of WrappedInstr { InputPort SI = SI; }

  ScanMux DR_MUX SelectedBy WIR[1:0] {2'b00 : WBY;
                                       2'b01 : WI1.SO;
                                       2'b10 : WI2.SO;
                                       2'b11 : WI3.SO;
  }

  ScanMux IR_MUX SelectedBy SWIR {1'b0 : DR_MUX;
                                  1'b1 : WIR[0];
  }
}

```



```
}  
}
```

SIT: SIT explicitly shows the hierarchy inside the wrapper. <SWIR Origin> would be the origin of the SWIR control

```
CHAIN WSP1500  
{  
  REGISTER WIR 2 Bypass: "0b0"  
  LINKER IR_MUX Binary <SWIR Origin> 2  
  (  
    LINKER DR_MUX Binary WIR 4  
    (  
      REGISTER WBY 1 Bypass: "0b0"  
      Instance WI1 Of WrappedInstr  
      Instance WI2 Of WrappedInstr  
      Instance WI3 Of WrappedInstr  
    )  
    REGISTER WIR 2 Bypass: "0b00"  
  )  
}
```

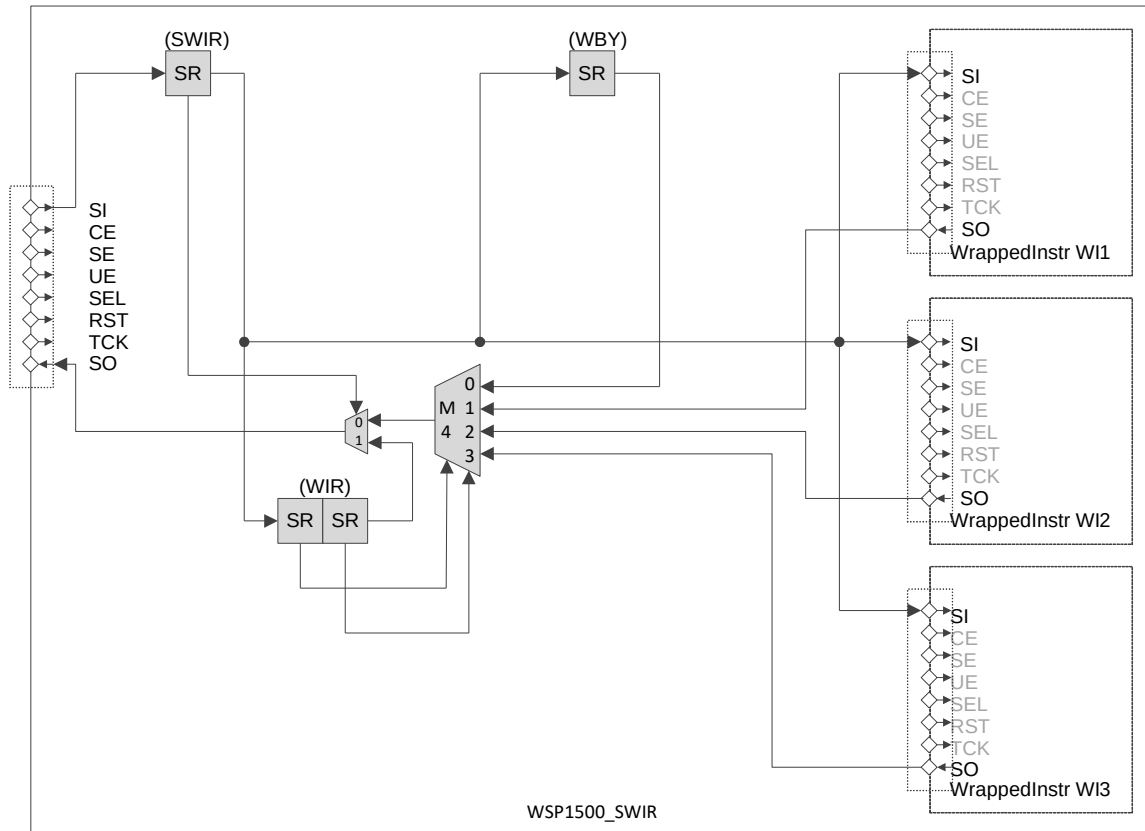


Figure 8.16—IEEE 1500 Wrapper Serial Port with Select WIR register

ICL:

```
Module WSP1500_SWIR {
    ScanInPort SI; CaptureEnPort CE; ShiftEnPort SE;
    UpdateEnPort UE; SelectPort SEL; ResetPort RST;
    TCKPort TCK; ScanOutPort SO { Source IR_MUX; }

    ScanRegister SWIR {ScanInSource SI; ResetValue 1'b1;}
    ScanRegister WBY {ScanInSource SWIR; ResetValue 1'b1;}
    Instance WIR Of SReg { InputPort SI = SWIR; Parameter Size = 2;}
}
```



```
Instance WI1 Of WrappedInstr { InputPort SI = SWIR; }

Instance WI2 Of WrappedInstr { InputPort SI = SWIR; }

Instance WI3 Of WrappedInstr { InputPort SI = SWIR; }


ScanMux DR_MUX SelectedBy WIR[1:0] {2'b00 : WBY;
                                     2'b01 : WI1.SO;
                                     2'b10 : WI2.SO;
                                     2'b11 : WI3.SO;
}

ScanMux IR_MUX SelectedBy SWIR {1'b0 : DR_MUX;
                                1'b1 : WIR[0];
}

}
```

SIT: SIT explicitly shows the hierarchy inside the wrapper

```
CHAIN WSP1500

{
  REGISTER SWIR 1 Bypass: "0b0"
  {
    REGISTER WIR 2 Bypass: "0b0"
    LINKER IR_MUX Binary SWIR 2
    (
      LINKER DR_MUX Binary WIR 4
      (
        REGISTER WBY 1 Bypass: "0b0"
        Instance WI1 Of WrappedInstr
        Instance WI2 Of WrappedInstr
```



```
)  
  
REGISTER WIR 2 Bypass: "0b00"  
  
)  
  
}  
  
}
```

SIT (compact): SIT provides a 1500 macro for easy instantiation

```
WRAPPER_1500 test_ WSP1500 3  
  
(  
  
  Instance WI1 Of WrappedInstr  
  
  Instance WI2 Of WrappedInstr  
  
  Instance WI3 Of WrappedInstr  
  
)
```

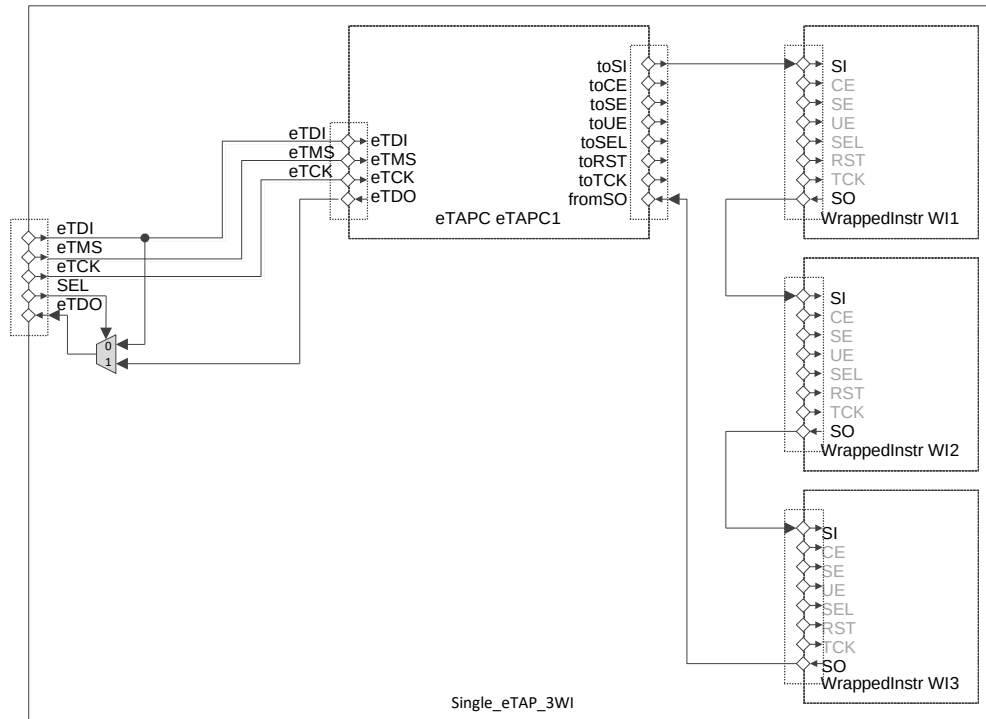


Figure 8.17—Embedded TAP Controller as an interface to instruments

ICL :

```
Module Single_eTAP_3WI {

  ScanInPort eTDI; TMSPort eTMS; DataInPort SEL;

  TCKPort eTCK; ScanOutPort eTDO { Source M2; }

  Instance eTAPC1 Of eTAPC { InputPort eTDI = eTDI;

                               InputPort eTMS = eTMS;

                               InputPort eTCK = eTCK;

                               InputPort fromSO = WI3.SO; }

  Instance WI1 Of WrappedInstr { InputPort SI = eTAPC1.toSI; }

  Instance WI2 Of WrappedInstr { InputPort SI = WI1.SO; }

  Instance WI3 Of WrappedInstr { InputPort SI = WI2.SO; }

  ScanMux M2 SelectedBy SEL {1'b0 : eTAPC1.eTDO;
```



```
1'b1 : eTDI;
```

```
    }  
}
```

```
Module eTAPC {  
    ScanInPort    eTDI;  
  
    ScanOutPort   eTDO { Source IRMux; }  
  
    ScanInPort    fromSO;  
  
    ScanOutPort   toSI {Source eTDI; }  
  
    TMSPort       eTMS;  
  
    TCKPort       eTCK;  
  
    ToSelectPort  en  { Source tdrEn0; }  
  
    ToResetPort   tlr { Source FSM.tlr; }  
  
    ToCaptureEnPort CE { Source FSM.CE; }  
  
    ToShiftEnPort SE { Source FSM.CE; }  
  
    ToUpdateEnPort UE { Source FSM.CE; }  
  
    Instance FSM Of TapStates {  
        InputPort tms = eTMS;  
  
        InputPort tck = eTCK;    }  
  
    ScanRegister IR[2:0] { ResetValue 3'b000;  
                           CaptureSource 3'bx01; }  
  
    ScanRegister Bypass { CaptureSource 1'b0;    }  
  
    ScanMux IRMux SelectedBy FSM.IRSel { 1'b0 : DRMux;  
                                           1'b1 : IR[0]; }  
  
    ScanMux DRMux SelectedBy IR[1:0] { 2'b00 : Bypass;  
                                           2'b01 : fromSO;    }  
  
    LogicSignal tdrEn0 { IR[1:0] == 2'b01; }  
  
    ScanInterface scan_host { Port toSI; Port en; Port fromSO; Port eTCK;
```



```
Port CE; Port SE; Port UE; Port tlr; }
```

```
}
```

```
Module TapStates {  
    TMSPort tms;  
    TCKPort tck;  
    ToResetPort tlr;  
    ToIRSelectPort IRSel;  
    ToCaptureEnPort CE;  
    ToShiftEnPort SE;  
    ToUpdateEnPort UE;  
}
```

SIT:

<Can be defined as a Translator>

8.22 Annex E, Example 22

Basic PDL for a simple instrument

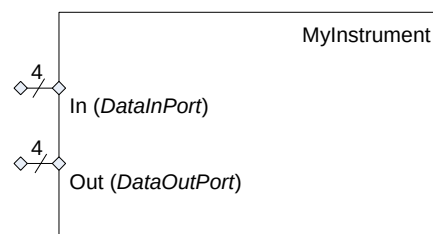


Figure 8.18—Simple instrument

ICL:

```
Module MyInstrument {
```



```
DataInPort In[3:0];  
  
DataOutPort Out[3:0];  
  
}
```

SIT: <Alias pour In et Out>

```
INSTRUMENT $In[3:0], $Out[3:0]
```

PDL:

```
iProcsForModule MyInstrument  
  
iProc ResetInstr {  
    iWrite In[0] 0;  
}  
  
iProc CheckInstr {  
    iWrite In 0x1; # note the use of hex ("0x" prefix)  
    iApply;  
    iRead Out;  
    iApply;  
}
```

C++ wrapped PDL:

```
void ResetInstr {  
    iWrite ("In[0]",0); <Alias TBD>  
}  
  
void iProc CheckInstr {  
    iWrite(In,0x1); # note the use of hex ("0x" prefix)  
    iApply();  
    iRead(Out);  
    iApply();  
}
```

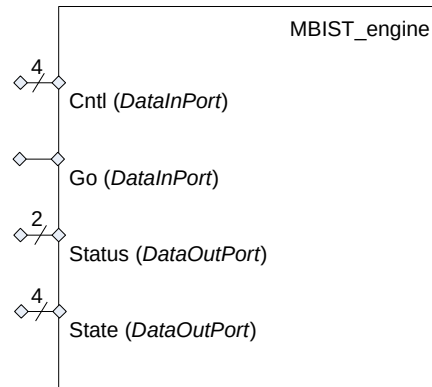


Figure 8.19—MBIST engine

ICL:

```

Module MBIST_engine {
    DataInPort Cntl[3:0];
    DataInPort Go;
    DataOutPort Status[1:0];
    DataOutPort State[3:0];
}

```

SIT:

```
CHAIN WSP1500
```

PDL:

```

iProc RunMBIST {Control,LoopCount} {
    iWrite Cntl $Control; # Control is passed in as a parameter
    iApply;
    iWrite Go 0b1;
    iApply;
    iRunLoop $LoopCount; # action command, like iApply
    iRead Status 0b11;
}

```



```
iApply;  
  
}  
  
iProc RunGalPatMBIST {  
  
    iCall RunMBIST 0b0111 32000;  
  
}
```

C++ wrapped PDL :

```
void RunMBIST (string Control, int LoopCount) {  
  
    iWrite ("Cntl",Control); # Control is passed in as a parameter  
  
    iApply();  
  
    iWrite("Go", "0b1");  
  
    iApply();  
  
    for (LoopCount)  
  
        iRead("Status",0b11);  
  
        iApply();  
  
}  
  
  
void RunGalPatMBIST {  
  
    RunMBIST("0b0111",32000);  
  
}
```

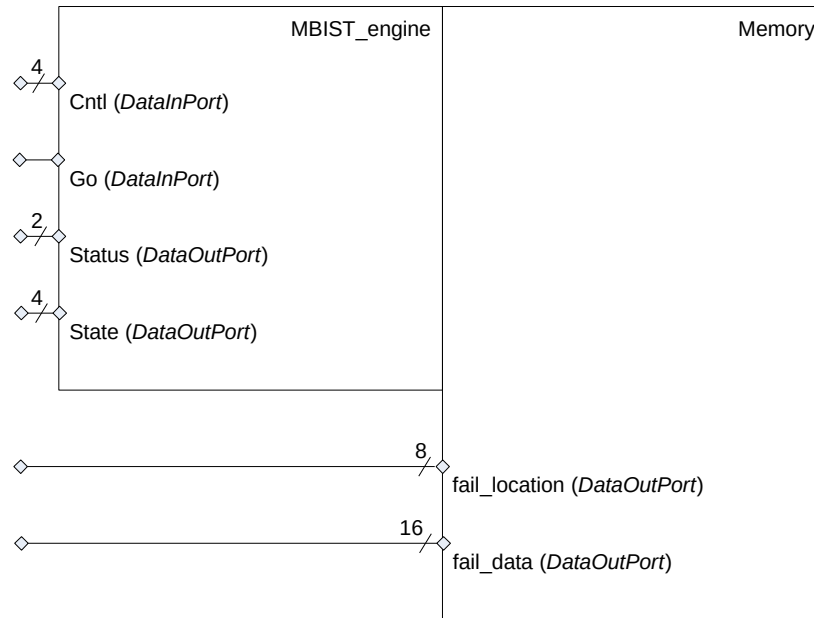


Figure 8.20— Separate Memory BIST and Memory modules

ICL :

```

Module MBIST_engine {
    DataInPort Cntl[3:0];
    DataInPort Go;
    DataOutPort Status[1:0];
    DataOutPort State[3:0];
}

```

```

Module Memory {
    DataOutPort fail_location[7:0];
    DataOutPort fail_data[15:0];
}

```

SIT:

<SIT starts from the register the instrument is connected to>



PDL:

```
iProc Collect_MBIST_failure_info {  
    iRead Status;  
  
    iRead State;  
  
    iRead fail_location;  
  
    iRead fail_data;  
  
    iApply;  
}
```

C++ wrapped PDL :

```
Void Collect_MBIST_failure_info {  
    iRead ("Status");  
  
    iRead ("State");  
  
    iRead ("fail_location");  
  
    iRead ("fail_data");  
  
    iApply();  
}
```

Alternative C++ wrapped PDL :

By exploiting C++ and PDL Level 1, data can be directly displayed and processed during execution:

```
bool Collect_MBIST_failure_info {  
    auto Status=iGet("Status");  
  
    iApply();  
  
    if (Status == MBIST_OK) return true; //No error, finish silently  
  
    //Error detect: collect and visualize failure data  
    auto State=iGet ("State");
```



```

auto fail_location=iGet ("fail_location");

auto fail_data=iGet ("fail_data");

iApply();

cerr << "MBIST ended in error: " << State <<

        fail_location << fail_data << endl;

return false;

}

```

8.25 Annex E, Example 25

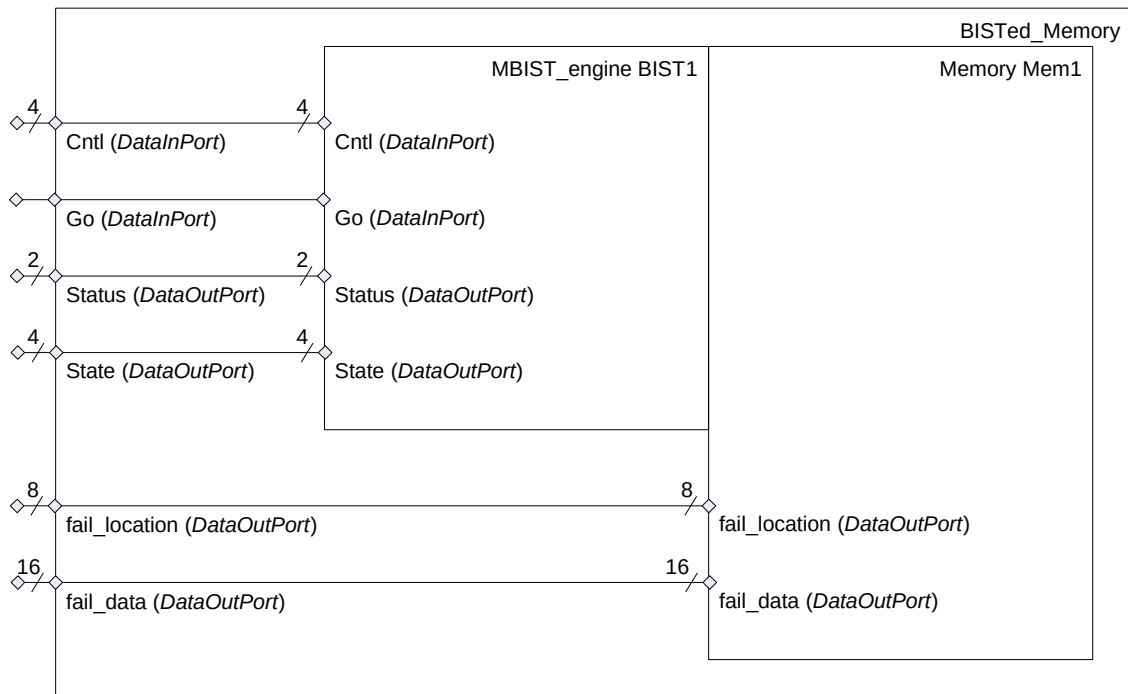


Figure 8.21—Combined MBIST and Memory modules

ICL :

```

Module BISTed_Memory {

    DataInPort Cntl[3:0];

    DataInPort Go;

    DataOutPort Status[1:0] {

```



```

DataOutPort State[3:0] {
    Source BIST1.State; }

DataOutPort fail_location[7:0] {
    Source Mem1.fail_location[7:0]; }

DataOutPort fail_data[15:0] {
    Source Mem1.fail_data[15:0]; }

Instance BIST1 Of MBIST_engine {
    InputPort Cntl = Cntl;
    InputPort Go = Go;
}

Instance Mem1 Of Memory;
}

```

SIT:

```

CHAIN BISTed_Memory
{ Instance Mem1 Of Memory;}

```

8.26 Annex E, Example 26

<Should be supported, did not have the opportunity to go through the ICL code>

ICL:

```

Module BlockA {
    DataInPort DH[7:0];
    DataInPort DL[7:0];
    DataOutPort QH[7:0] { Source Q[15:8] ; }
    DataOutPort QL[7:0] { Source Q[7:0] ; }
    AddressPort AH[19:16];
}

```



```
AddressPort AL[15:0];

ReadEnPort RE; //One and only one RE/WE ports on

WriteEnPort WE; //modules containing addressable regs

OneHotDataGroup Q[15:0] {

    DataRegister R0[15:0] {

        AddressValue 8'd0;

    }

    DataRegister R1[15:0] {

        AddressValue 8'd1;

    }

    ...

    DataRegister R72[15:0] {

        AddressValue 8'd72;

    }

}

Alias Ena = R0[15];

Alias AMode[8:0] = R0[14:6] {

    RefEnum AModes;

}

...

Alias RH = R72[0];

Enum AModes { M1 = 9'h0; ...; M9 = 9'h0A4; }

}

Module BlockB {

    DataInPort D[15:0];

    DataOutPort Q[15:0] { Source QM[15:0]; }

    AddressPort A[19:0];

    ReadEnPort ReadEn;
```



```
OneHotDataGroup QM[15:0] {  
    Instance BlockA_I1 Of BlockA {  
        AddressValue 16'h0000;  
    }  
    Instance BlockA_I2 Of BlockA {  
        AddressValue 16'h2300;  
    }  
    ...  
}  
  
// Default DataIn mapping is right justified  
// Can be explicitly specified with OneHotDataGroup syntax:  
OneHotDataGroup Q[15:0] {  
    Port BlockA_I1.DOH[7:0];  
    Port BlockA_I1.DOL[0:7];  
    ...  
}
```

SIT: a OneHotData group is simply a PathSelector for a Mux, as explained in Section 3

```
CHAIN BlockA  
  
{  
    ...  
    ...  
}
```

<Need to add description of the Black Box SIT syntax>

8.28 Annex E, Example 28

Wide scan interface example

<DEPRECATED>

<Exemple parfait de ICL et PDL faisant ce qu'ils ne devraient jamais faire, c'est à dire mélanger opération internes et transactions sur l'interface. A interdire !>

8.29 Annex E, Example 29

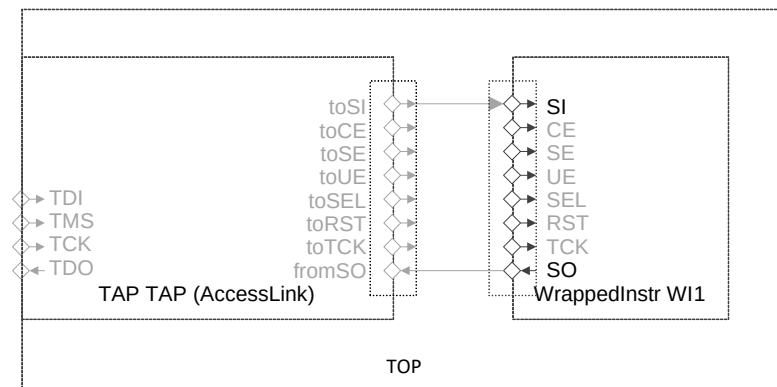


Figure 8.22—Simple IEEE 1149.1 AccessLink example

ICL:

```
Module TOP {
    Instance WI1 Of WrappedInstr { }
    AccessLink TAP Of STD_1149_1_2001 {
        BSDLEntity TOP;
        ijtag_en { // Name of instruction
            ScanInterface { WI1.scan_client; } // ScanInterface
        }
    }
}
```



SIT (compact): the macro JTAG_TAP can be used, by giving it the IR size and number of chains. IR coding is inferred as binary

```
JTAG_TAP TOP $IR_SIZE $N_CHAINS  
  
(  
    Instance W11 Of WrappedInstr  
  
)
```

SIT (verbose): as explained in [2], SIT implements the "AccessInterface" abstraction, which allows the IR register and decode to be modelled in 1687 terms.

```
TRANSLATOR top Emulation  
  
ACCESS_INTERFACE TAP SVF_Raw  
  
(  
    Instance W11 Of WrappedInstr  
  
)
```

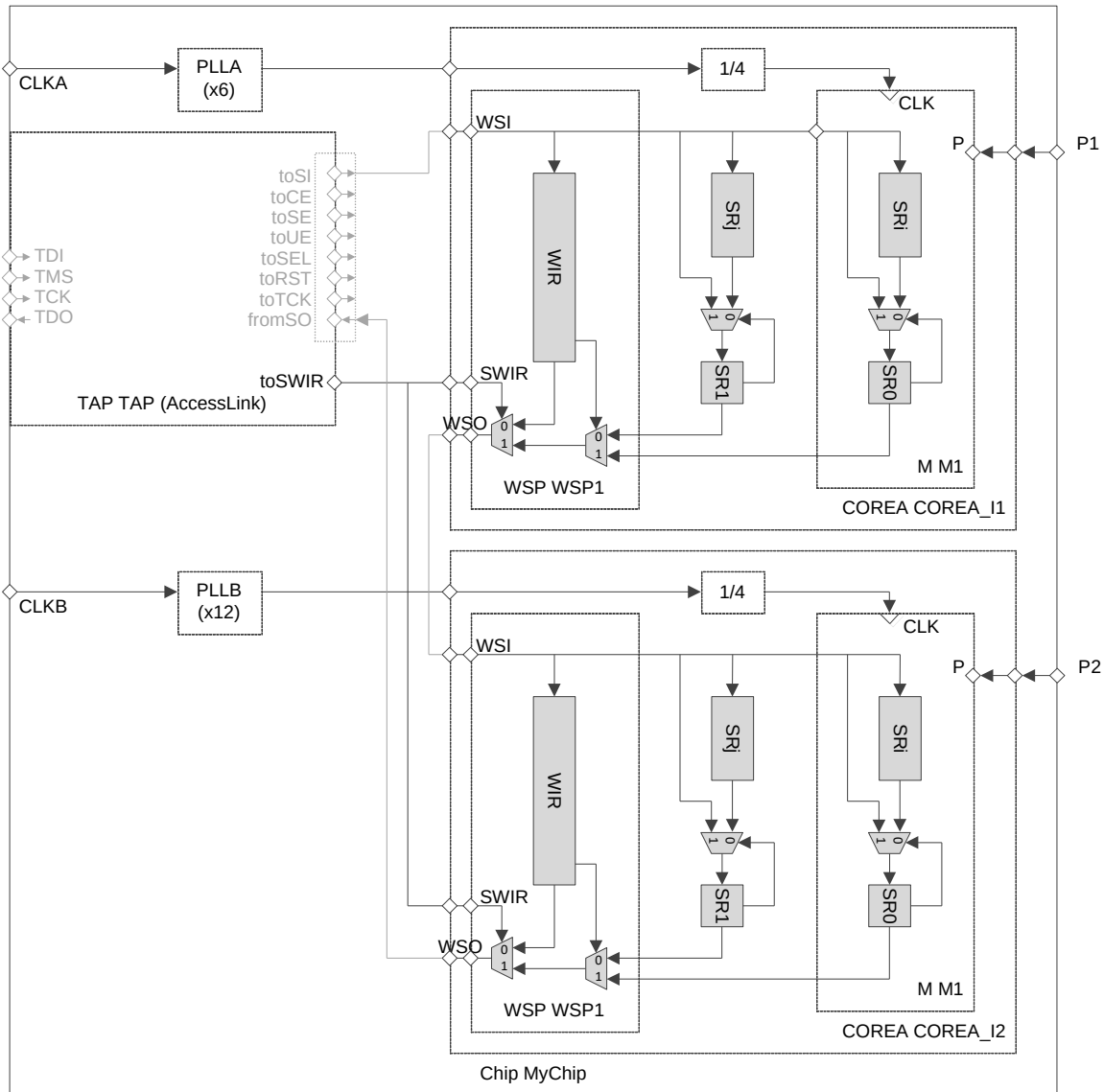


Figure 8.23—Complex IEEE 1149.1 AccessLink example

ICL :

```
Module MyChip {
    ClockPort CLKA;
    ClockPort CLKB;
    DataInPort P1;
    DataInPort P2;
```



```
Instance PLLA_I1 Of PLLA { InputPort ref = CLKA;}

Instance PLLB_I1 Of PLLB { InputPort ref = CLKB;}

Instance COREA_I1 Of COREA {

    InputPort P = P1;

    InputPort CLK = PLLA_I1.vco;

    InputPort SWIR = Tap1.toSWIR;

}

Instance COREA_I2 Of COREA {

    InputPort P = P2;

    InputPort CLK = PLLB_I1.vco;

    InputPort SWIR = Tap1.toSWIR;

}

AccessLink Tap1  Of STD_1149_1 {

    BSDL_Entity Mychip;

    wir_select { ScanInterface { COREA_I1.SP1;

                                COREA_I2.SP1;}}

    ActiveSignals { toSWIR ;}

}

    wdr_select { ScanInterface {COREA_I1;

                                COREA_I2; }

    // Because COREA only has one scan port, explicitly listing
    the ScanInterface (".SP1") is optional

    // Also note that since "toSWIR" is not listed as an
    ActiveSignal, so it is INACTIVE here.

}

}

}
```



```
Module PLLA {  
    ClockPort ref;  
    ToClockPort vco { FreqMultiplier: 6; }  
}  
  
Module PLLB {  
    ClockPort ref;  
    ToClockPort vco { FreqMultiplier: 12; }  
}  
  
Module CoreA {  
    ClockPort CLK;  
    ScanInPort WSI;  
    ScanOutPort WSO {Source : WTAP.WSO; }  
    ShiftEnPort shiftWR;  
    CaptureEnPort captureWR;  
    UpdateEnPort updateWR;  
    ResetPort WRST;  
    TCKPort WRCK;  
    DataInPort SWIR;  
    ScanInterface SP1 { Port WSI; Port WSO; Port SEL; Port WSI; Port WSO;  
                        Port shiftWR; Port captureWR; Port updateWR;  
    }  
    . . .  
}  
  
From BSDL File:  
  
Entity Mychip  
  
attribute REGISTER_OPCODE = " wir_select (110)," &  
                            " wdr_select (111);"
```



SIT:

<TDB>

CHAIN WSP1500

8.31 Annex E, Example 31

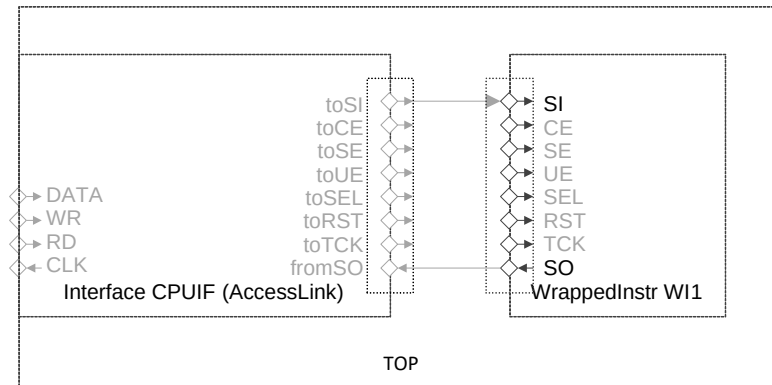


Figure 8.24—Generic AccessLink example

ICL:

```
Module chip {
    Instance WI1 Of WrappedInstr { }

    AccessLink CPUIF of Interface {
        ScanInterface WI1.scan_client;

        WordSize 8; // 6+2

        CPU_pins {DATA, WR, RD, CLK}
    }
}
```

SIT: a generic AccessLink (or better to say, Access Interface) can be added to MAST as explained in Section 6, and then be directly instantiated

```
ACCESS_INTERFACE CPUIF Interface "8 DATA WR RD CLK"{
    (
        Instance WI1 Of WrappedInstr
```





9 References

- [1] Portolan Michele, "A novel test generation and application flow for functional access to IEEE 1687 instruments", Test Symposium (ETS), 2016 21th IEEE European, 23-27 May 2016, DOI: [10.1109/ETS.2016.7519302](https://doi.org/10.1109/ETS.2016.7519302)
- [2] Portolan Michele, "Accessing 1687 Systems Using Arbitrary Protocols", Test Conference (ITC), 2016 IEEE International, November 2016
- [3] M. Portolan, "The Automated Test Flow, the Present and the Future", IEEE Transactions on Computer-Aided Design (TCAD), DOI: 10.1109/TCAD.2019.2961328, December 2019